Machine Learning
CMPSCI 589

Bruno C. da Silva
bsilva@cs.umass.edu

Neural Networks (pt. 4)

Numerically Checking Gradients
Heuristics to Accelerate Convergence

UMassAmherst
College of Information & Computer Sciences

Checking the Correctness of your Implementation via Gradient Checking

(i.e., numerically checking the derivatives computed by back propagation)

Backpropagation - Updating Weights

$$\delta_1^{l=L} = \left(f_\theta(x) - y\right)$$

$$\delta_i^{l=k} = \left(\sum_{j=1}^{N} \theta_{ij}^{l=k+1} \ \delta_j^{l=k+1}\right)\left(a_i^{l=k}\right)\left(1 - a_i^{l=k}\right)$$

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \ \delta_i^{l=k+1}$$

$$\frac{\partial J}{\partial \theta_{11}^{l=1}} \qquad \theta_{11}^{l=1} := \theta_{11}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=1}}$$

$$\frac{\partial J}{\partial \theta_{12}^{l=1}} \qquad \theta_{12}^{l=1} := \theta_{12}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{12}^{l=1}}$$

$$\vdots \qquad\qquad \vdots$$

$$\frac{\partial J}{\partial \theta_{11}^{l=2}} \qquad \theta_{11}^{l=2} := \theta_{11}^{l=2} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=2}}$$

$$\vdots \qquad\qquad \vdots$$

$$\frac{\partial J}{\partial \theta_{11}^{l=L}} \qquad \theta_{11}^{l=L} := \theta_{11}^{l=L} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=L}}$$

Complex implementation - Easy to make mistakes

Gradients above = *rate of change* of the cost function J with respect to each weight θ

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \ldots, \theta_{11}^{l=L}, \ldots) = \frac{1}{n}\sum_{i=1}^{n} -y^{(i)}\left(-\log(f_\theta(x^{(i)})\right) - (1-y^{(i)})\left(-\log(1-f_\theta(x^{(i)}))\right)$$

Backpropagation - Updating Weights

Complex implementation - easy to make mistakes

Gradients above = *rate of change* of the cost function J with respect to each weight θ

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \ldots, \theta_{11}^{l=L}, \ldots) = \frac{1}{n}\sum_{i=1}^{n} -y^{(i)}\Big(-\log(f_\theta(x^{(i)})\Big) - (1-y^{(i)})\Big(-\log(1-f_\theta(x^{(i)}))\Big)$$

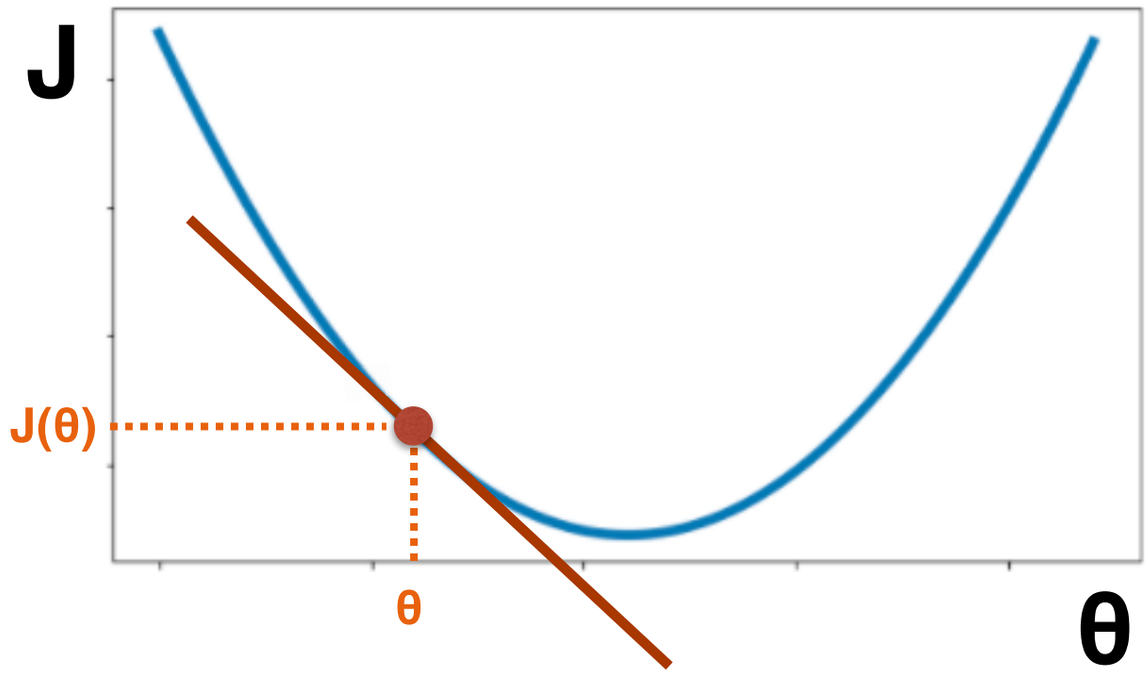
We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm



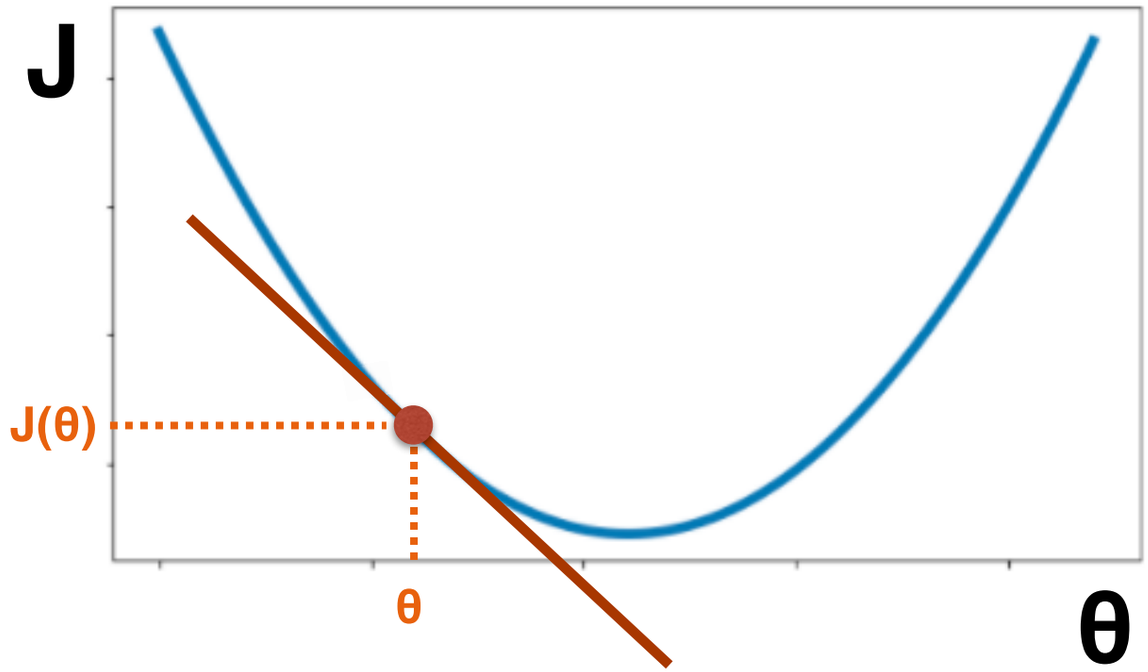
$$\frac{\partial J(\theta)}{\partial \theta} = \text{rate of change of the function J}$$

How much does J change after an infinitesimal change to θ

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm



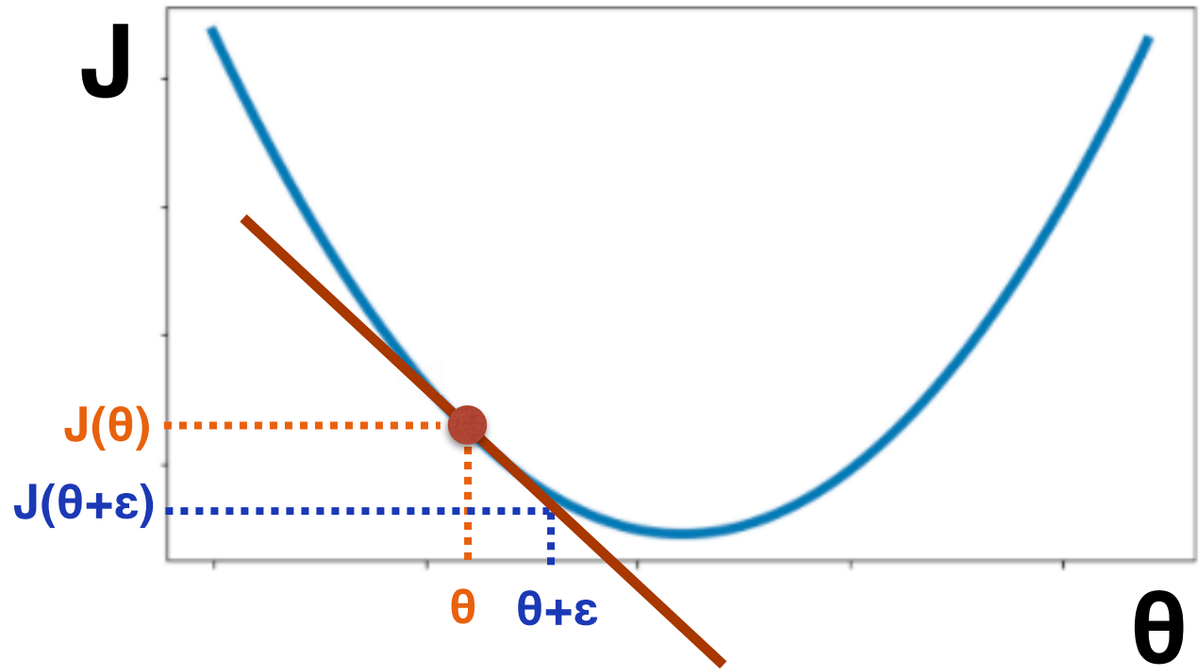
$$\frac{\partial J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

How much does J change after an infinitesimal change to θ

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm



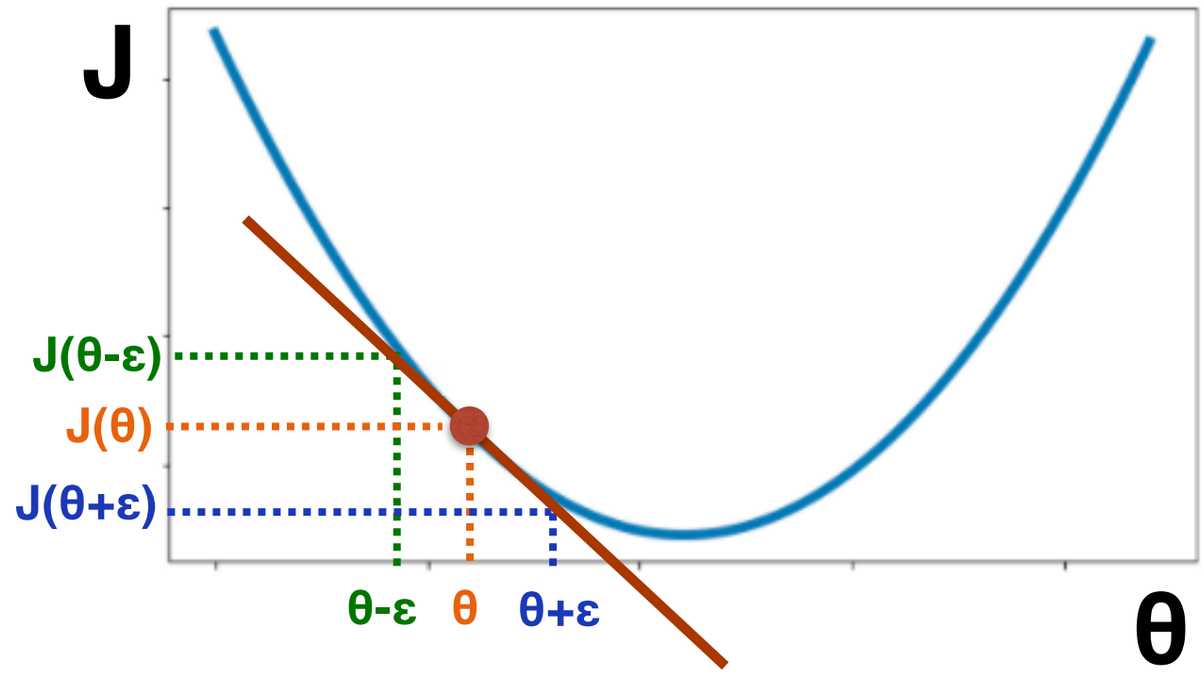
$$\frac{\partial J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

How much does J change after an infinitesimal change to θ

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm



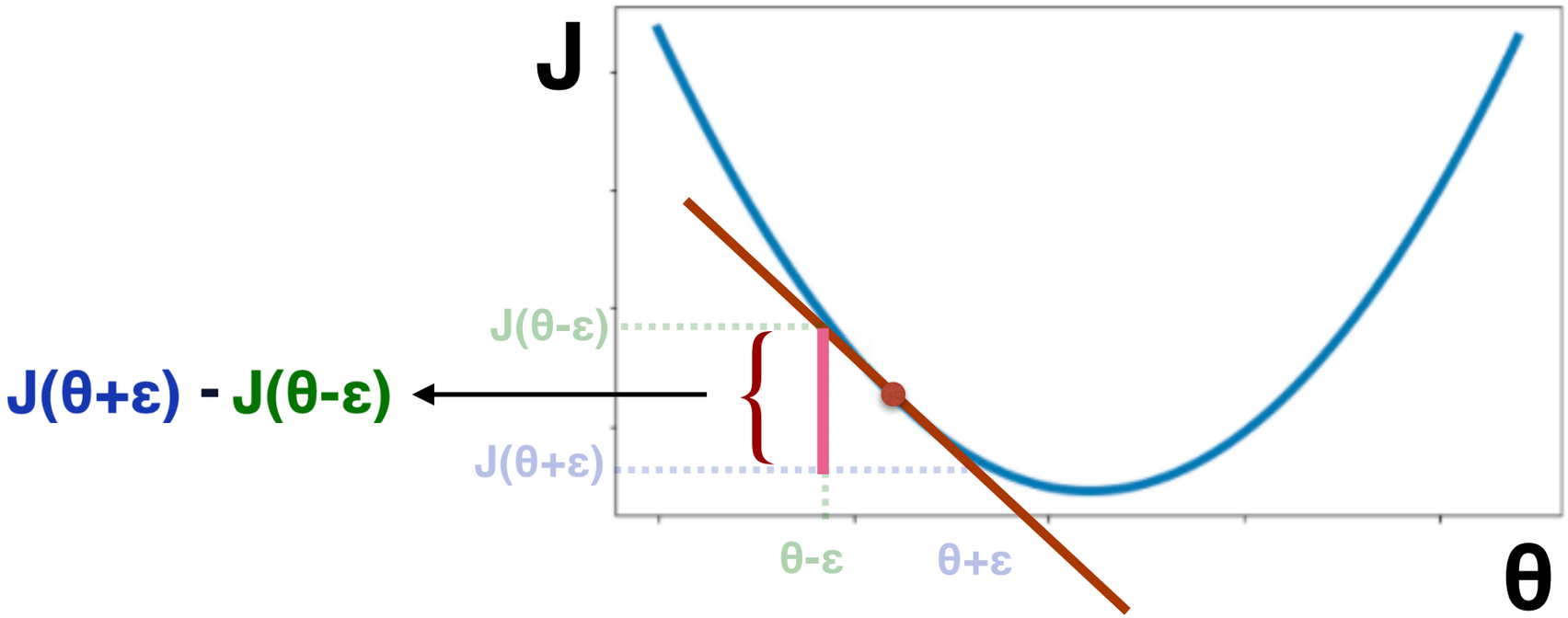
$$\frac{\partial J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

How much does J change after an infinitesimal change to θ

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm



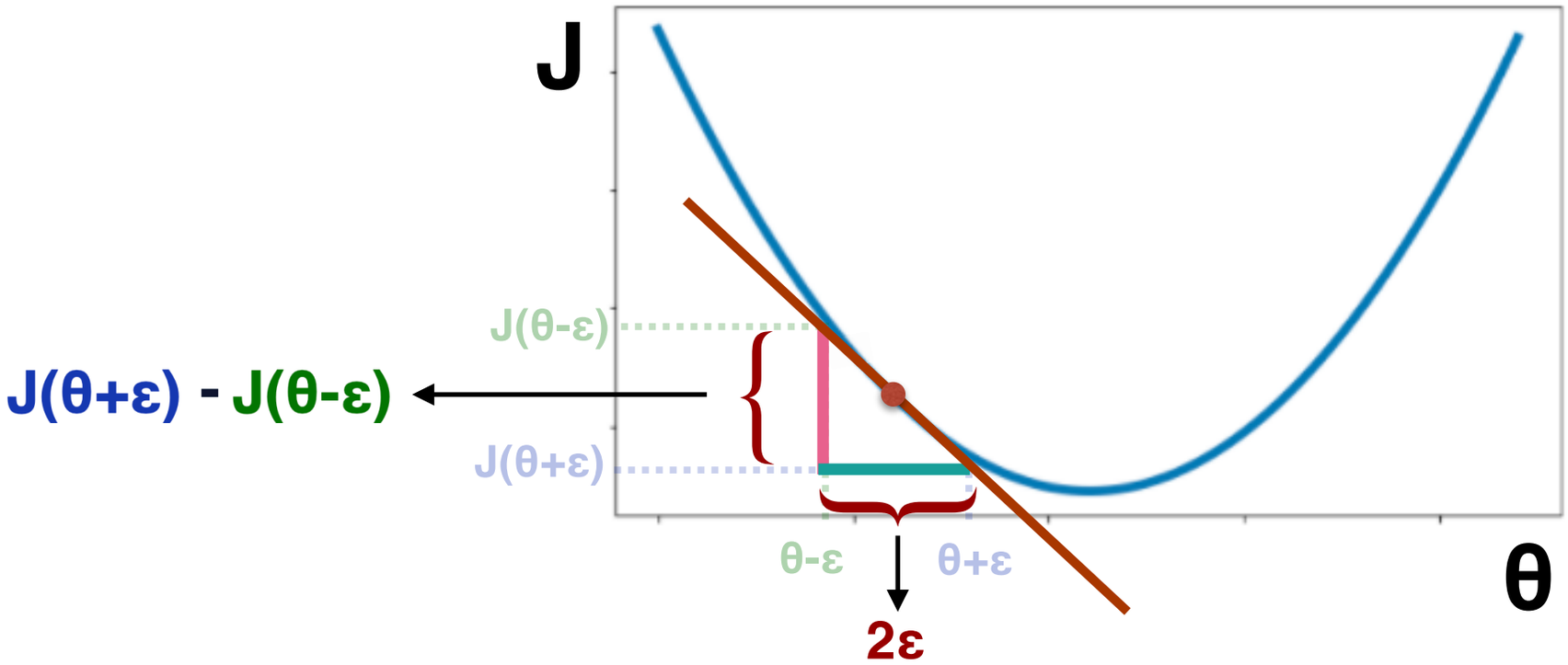
$$\frac{\partial J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

How much does J change after an infinitesimal change to θ

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm



$$\frac{\partial J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

How much does J change after an infinitesimal change to θ

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm

$$\frac{\partial\ J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

$$\frac{\partial}{\partial \theta_1} J(\theta_1, \theta_2, \ldots, \theta_K) = \frac{J(\theta_1 + \varepsilon, \theta_2, \ldots, \theta_K) - J(\theta_1 - \varepsilon, \theta_2, \ldots, \theta_K)}{2\varepsilon}$$

(Using a small value, ε. For instance, ε = 0.01)

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm

$$\frac{\partial\ J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

$$\frac{\partial}{\partial \theta_1} J(\theta_1, \theta_2, \ldots, \theta_K) = \frac{J(\theta_1 + \varepsilon, \theta_2, \ldots, \theta_K) - J(\theta_1 - \varepsilon, \theta_2, \ldots, \theta_K)}{2\varepsilon}$$

(Using a small value, ε. For instance, ε = 0.01)

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm

$$\frac{\partial}{\partial \theta} J(\theta) \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

$$\frac{\partial}{\partial \theta_1} J(\theta_1, \theta_2, \ldots, \theta_K) = \frac{J(\theta_1 + \varepsilon, \theta_2, \ldots, \theta_K) - J(\theta_1 - \varepsilon, \theta_2, \ldots, \theta_K)}{2\varepsilon}$$

$$\frac{\partial}{\partial \theta_2} J(\theta_1, \theta_2, \ldots, \theta_K) = \frac{J(\theta_1, \theta_2 + \varepsilon, \ldots, \theta_K) - J(\theta_1, \theta_2 - \varepsilon, \ldots, \theta_K)}{2\varepsilon}$$

(Using a small value, ε. For instance, ε = 0.01)

Backpropagation - Updating Weights

We can *numerically estimate* the gradient $\frac{\partial J}{\partial \theta_{ij}^{l=k}}$ with respect to each weight $\theta_{ij}^{l=k}$

And compare them with the gradients computes by the backpropagation algorithm

$$\frac{\partial\ J(\theta)}{\partial \theta} \approx \frac{J(\theta+\varepsilon) - J(\theta-\varepsilon)}{2\varepsilon}$$

$$\begin{aligned}
\frac{\partial}{\partial \theta_1} J(\theta_1, \theta_2, \ldots, \theta_K) &= \frac{J(\theta_1 + \varepsilon, \theta_2, \ldots, \theta_K) - J(\theta_1 - \varepsilon, \theta_2, \ldots, \theta_K)}{2\varepsilon} \\
\frac{\partial}{\partial \theta_2} J(\theta_1, \theta_2, \ldots, \theta_K) &= \frac{J(\theta_1, \theta_2 + \varepsilon, \ldots, \theta_K) - J(\theta_1, \theta_2 - \varepsilon, \ldots, \theta_K)}{2\varepsilon} \\
&\vdots \\
\frac{\partial}{\partial \theta_K} J(\theta_1, \theta_2, \ldots, \theta_K) &= \frac{J(\theta_1, \theta_2, \ldots, \theta_K + \varepsilon) - J(\theta_1, \theta_2, \ldots, \theta_K - \varepsilon)}{2\varepsilon}
\end{aligned}$$

(Using a small value, ε. For instance, ε = 0.01)

Numerically Checking Gradients

Example: numerically estimating the gradient of the following function:

$$f(x) = x^2 + 4.5x^3 - \sin(x)$$

at x=0.7 (and using ε = 0.01)

$$\frac{\partial f(x)}{\partial x} \approx \frac{f(x+\varepsilon) - f(x-\varepsilon)}{2\varepsilon}$$

(demo)

Numerically Checking Gradients

Example: numerically estimating the gradient of the following function:

$$f(x) = x^2 + 4.5x^3 - \sin(x)$$

at x=0.7 (and using ε = 0.01)

$$\frac{\partial f(x)}{\partial x} \approx \frac{\left((0.7+0.01)^2 + 4.5 \times (0.7+0.01)^3 - \sin(0.7+0.01)\right) - \left((0.7-0.01)^2 + 4.5 \times (0.7-0.01)^3 - \sin(0.7-0.01)\right)}{2 \times 0.01}$$

= 7.251

Let us compare that with the real/correct value of the gradient (derivative) of this function:

$$\frac{\partial f(x)}{\partial x} = 2x + 4.5 \times 3 \times x^2 - \cos(x)$$

at x=0.7

$$= 2 \times 0.7 + 4.5 \times 3 \times 0.7^2 - \cos(0.7)$$

= 7.250

How to check if your implementation of backpropagation is correct

Compare the gradients of J computed via the backpropagation algorithm:

$$\delta_1^{l=L} = \left(f_\theta(x) - y\right) \qquad \delta_i^{l=k} = \left(\sum_{j=1}^{N} \theta_{ij}^{l=k+1} \ \delta_j^{l=k+1}\right)\left(a_i^{l=k}\right)\left(1 - a_i^{l=k}\right)$$

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \ \delta_i^{l=k+1}$$

with numerical estimated of the gradient of J

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \ldots, \theta_{11}^{l=L}, \ldots) = \frac{1}{n}\sum_{i=1}^{n} -y^{(i)}\left(-\log(f_\theta(x^{(i)})\right) - (1-y^{(i)})\left(-\log(1-f_\theta(x^{(i)}))\right)$$

For instance: $$\frac{\partial}{\partial \theta_1} J(\theta_1, \theta_2, \ldots, \theta_K) = \frac{J(\theta_1 + \varepsilon, \theta_2, \ldots, \theta_K) - J(\theta_1 - \varepsilon, \theta_2, \ldots, \theta_K)}{2\varepsilon}$$

How to check if your implementation of backpropagation is correct

Compare the gradients of J computed via the backpropagation algorithm:

$$\delta_1^{l=L} = \left(f_\theta(x) - y\right) \qquad \delta_i^{l=k} = \left(\sum_{j=1}^{N} \theta_{ij}^{l=k+1} \ \delta_j^{l=k+1}\right)\left(a_i^{l=k}\right)\left(1 - a_i^{l=k}\right)$$

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \ \delta_i^{l=k+1}$$

with numerical estimated of the gradient of J

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \ldots, \theta_{11}^{l=L}, \ldots) = \frac{1}{n}\sum_{i=1}^{n} -y^{(i)}\left(-\log(f_\theta(x^{(i)})\right) - (1-y^{(i)})\left(-\log(1-f_\theta(x^{(i)}))\right)$$

For instance: $$\frac{\partial}{\partial \theta_1} J(\theta_1, \theta_2, \ldots, \theta_K) = \frac{J(\theta_1 + \varepsilon, \theta_2, \ldots, \theta_K) - J(\theta_1 - \varepsilon, \theta_2, \ldots, \theta_K)}{2\varepsilon}$$

Review: Backpropagation

Repeat until stopping criterion is met

1. For each training instance $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$:
   - 1.1. Propagate $\mathbf{x}^{(i)}$ and compute each of the network's outputs, $\mathbf{f}_\theta(\mathbf{x}^{(i)})$
   - 1.2. $\delta^{(l=L)} = \mathbf{f}_\theta(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)}$ // computes the delta values of all output neurons
   - 1.3. For each network layer, k=L-1…2 // computes the delta values of all neurons in the hidden layers
     - $\delta^{(l=k)} = [\theta^{(l=k)}]^T \delta^{(l=k+1)} .* a^{(l=k)} .* (1-a^{(l=k)})$
     - Removes the first element of $\delta^{(l=k)}$ (i.e., the delta value associated with the bias neuron of the k-th layer of the network)
   - 1.4. For each network layer, k=L-1…1 // updates the gradients of the weights of each layer, based on the current training instance
     - $D^{(l=k)} = D^{(l=k)} + \delta^{(l=k+1)} [a^{(l=k)}]^T$ // accumulates, in $D^{(l=k)}$, the gradients computed based on the current training instance
2. For each network layer, k=L-1…1 // computes the final (regularized) gradients of the weights of each layer
   - 2.1. Let $P^{(l=k)}$ be equal to $(\lambda .* \theta^{(l=k)})$, and set its first column to be all zeros // computes the regularizer, λ, of all non-bias weights
   - 2.2. $D^{(l=k)} = (1/n) (D^{(l=k)} + P^{(l=k)})$ // combines gradients with their regularization terms; divides by #instances to obtain average gradient
3. // At this point, $\mathbf{D}^{(l=1)}$ contains the gradients of the weights $\boldsymbol{\theta}^{(l=1)}$; (…); and $\mathbf{D}^{(l=L-1)}$ contains the gradients of the weights $\boldsymbol{\theta}^{(l=L-1)}$
4. For each network layer, k=L-1…1 // updates the weights of each layer based on their corresponding gradients
   - 4.1. $\boldsymbol{\theta}^{(l=k)} = \boldsymbol{\theta}^{(l=k)} - \alpha .* \mathbf{D}^{(l=k)}$

Backpropagation

Repeat until stopping criterion is met

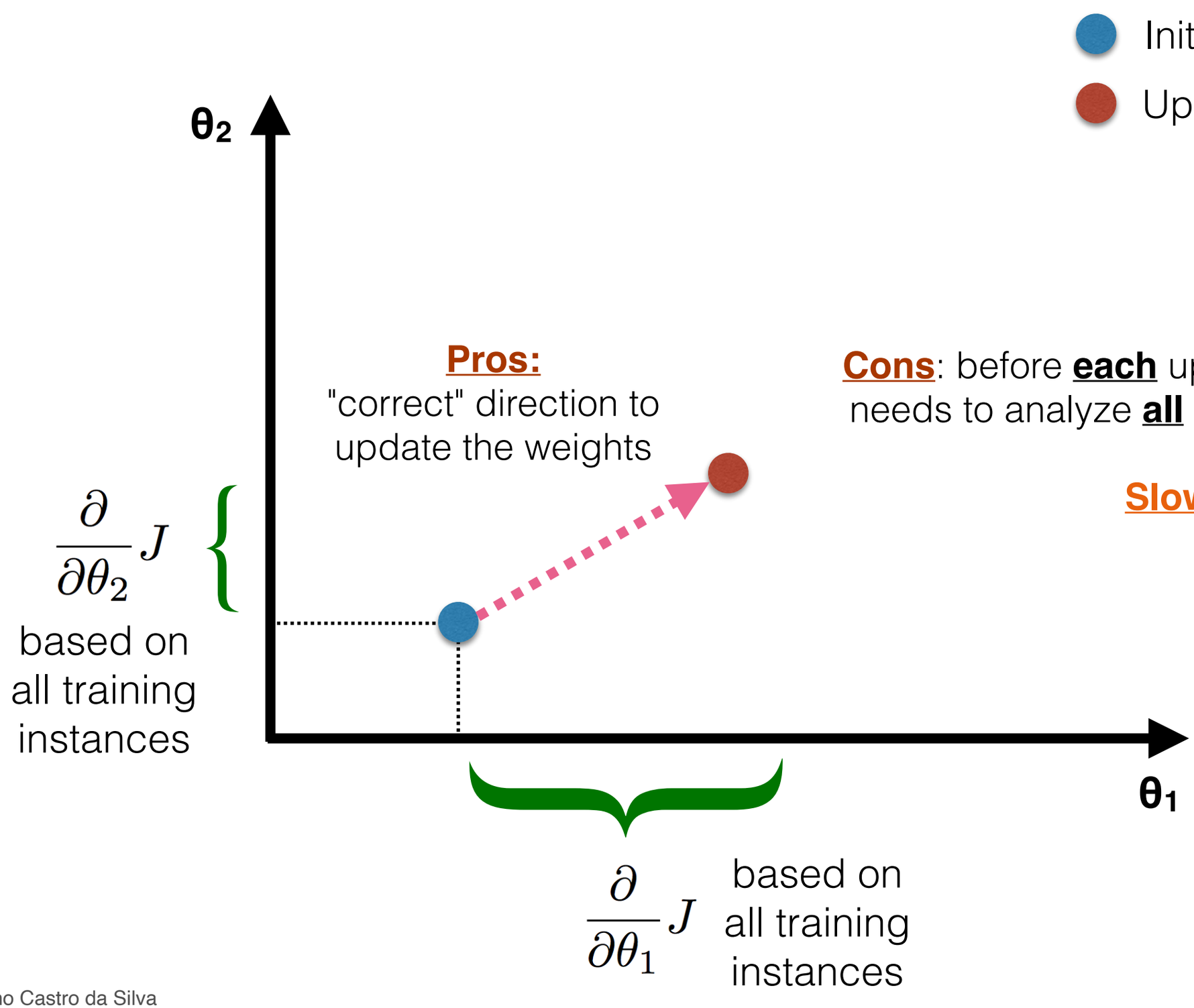
1. For each training instance $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$:
   - 1.1. Propagate $\mathbf{x}^{(i)}$ and compute each of the network's outputs, $\mathbf{f}_\theta(\mathbf{x}^{(i)})$
   - 1.2. $\delta^{(l=L)} = \mathbf{f}_\theta(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)}$ // computes the delta values of all output neurons
   - 1.3. For each network layer, k=L-1…2 // computes the delta values of all neurons in the hidden layers
     - $\delta^{(l=k)} = [\theta^{(l=k)}]^T \delta^{(l=k+1)} \; .* \; a'(z^{(l=k)})$
     - Removes the first element of $\delta^{(l=k)}$ (i.e., the delta value associated with the bias neuron of the k-th layer of the network)
   - 1.4. For each network layer, k=L-1…1 // updates the gradients of the weights of each layer, based on the current training instance
     - $\mathbf{D}^{(l=k)} = \mathbf{D}^{(l=k)} + \delta^{(l=k+1)} [a^{(l=k)}]^T$ // the gradients $\mathbf{D}^{(l=k)}$ are computed based on the current training instance
2. For each network layer, k=L-1…1 // computes the final (regularized) gradients of the weights of each layer
   - 2.1. Let $\mathbf{P}^{(l=k)}$ be equal to $(\lambda \; .* \; \theta^{(l=k)})$, and set its first column to be all zeros // computes the regularizer, λ, of all non-bias weights
   - 2.2. $\mathbf{D}^{(l=k)} = (1/n) (\mathbf{D}^{(l=k)} + \mathbf{P}^{(l=k)})$ // combines gradients with their regularization terms; divides by #instances to obtain average gradient
3. // At this point, $\mathbf{D}^{(l=1)}$ contains the gradients of the weights $\theta^{(l=1)}$; (…); and $\mathbf{D}^{(l=L-1)}$ contains the gradients of the weights $\theta^{(l=L-1)}$
4. For each network layer, k=L-1…1 // updates the weights of each layer based on their corresponding gradients
   - 4.1. $\theta^{(l=k)} = \theta^{(l=k)} - \alpha \; .* \; \mathbf{D}^{(l=k)}$

. . .

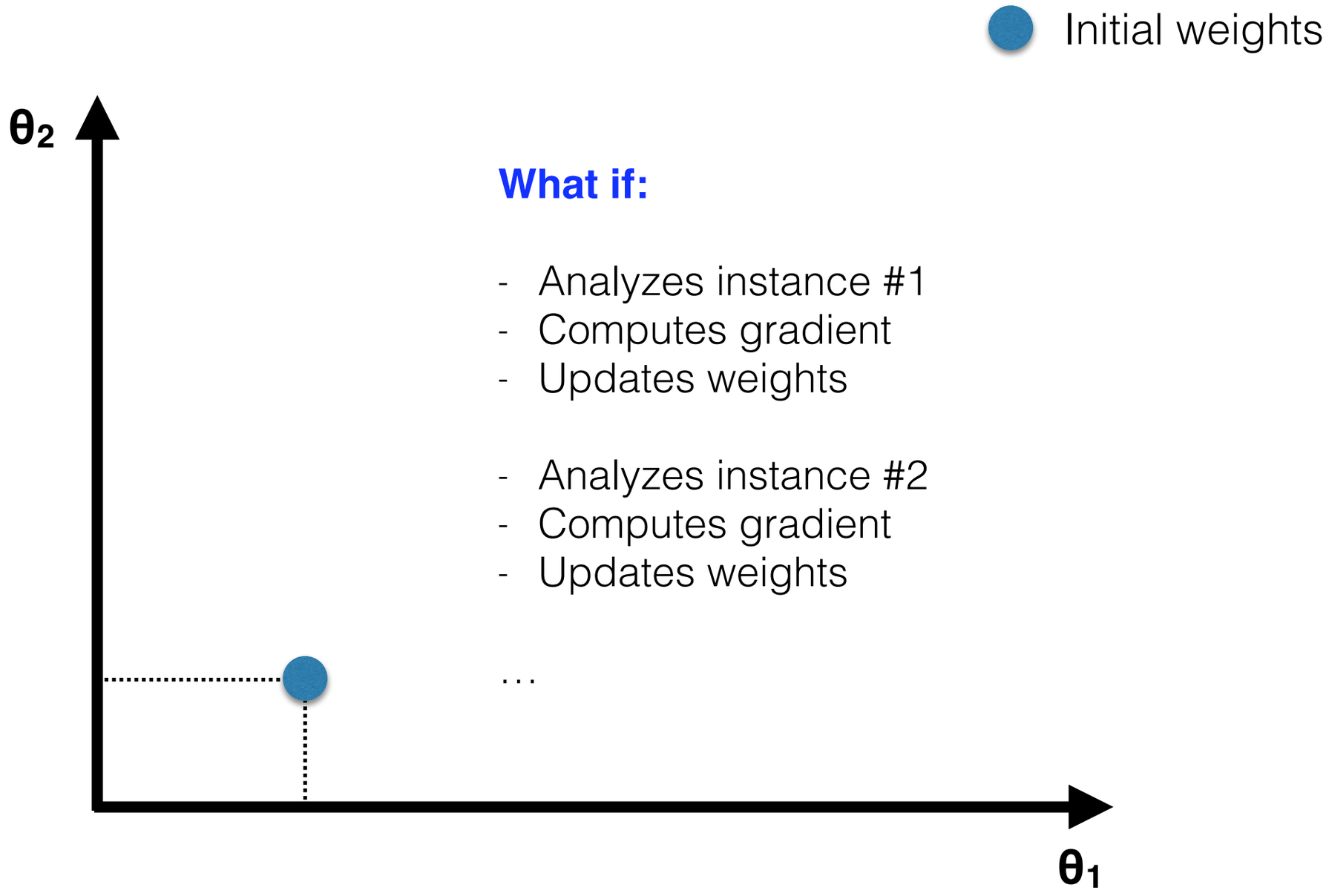
Updates weights once only after analyzing all training examples

Batch Gradient Descent

Batch Gradient Descent



Stochastic Gradient Descent



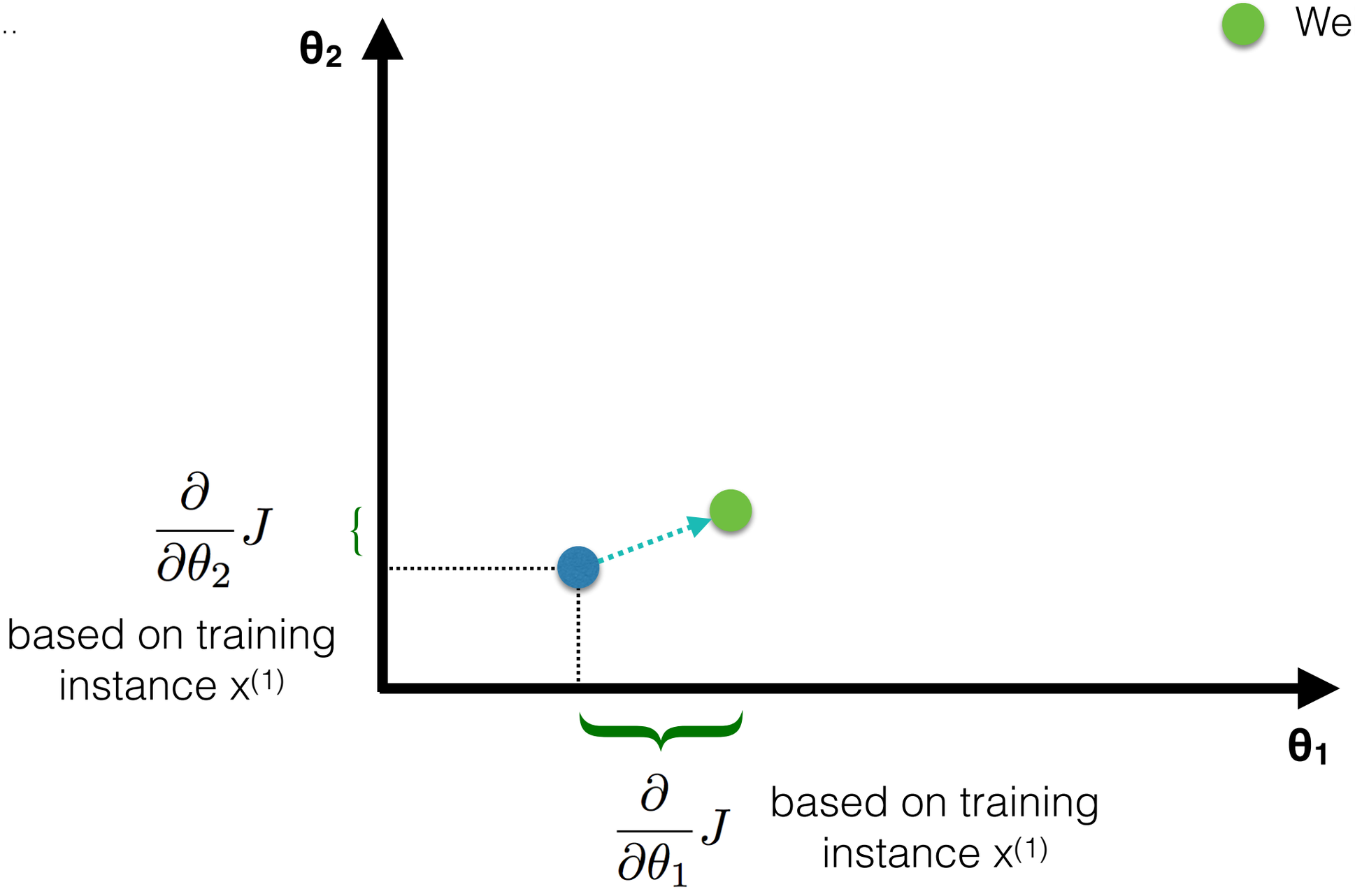
What if:

- Analyzes instance #1
- Computes gradient
- Updates weights

- Analyzes instance #2
- Computes gradient
- Updates weights

…

Stochastic Gradient Descent



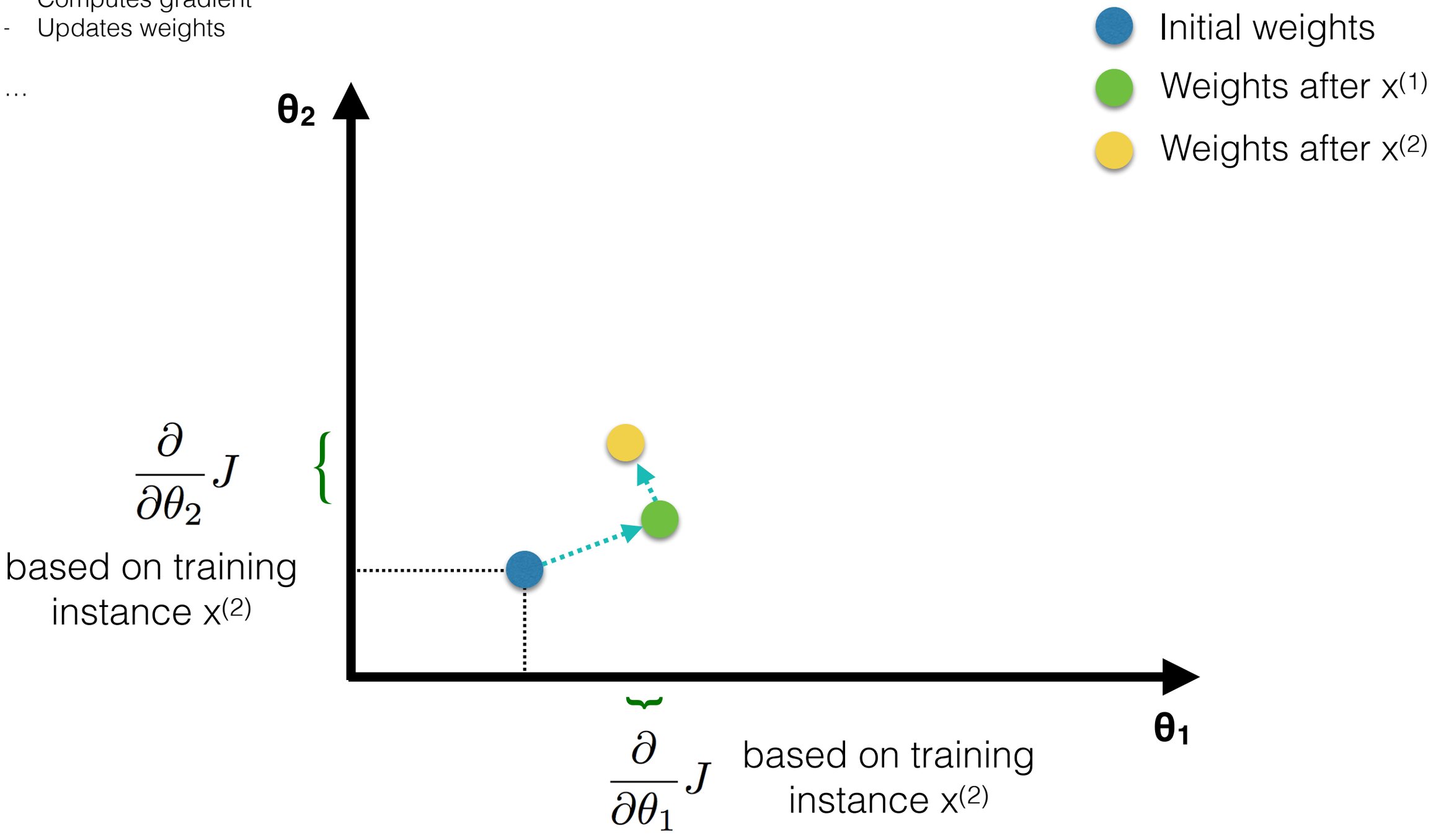
What if:

- Analyzes instance #1
- Computes gradient
- Updates weights

- Analyzes instance #2
- Computes gradient
- Updates weights

…

Stochastic Gradient Descent



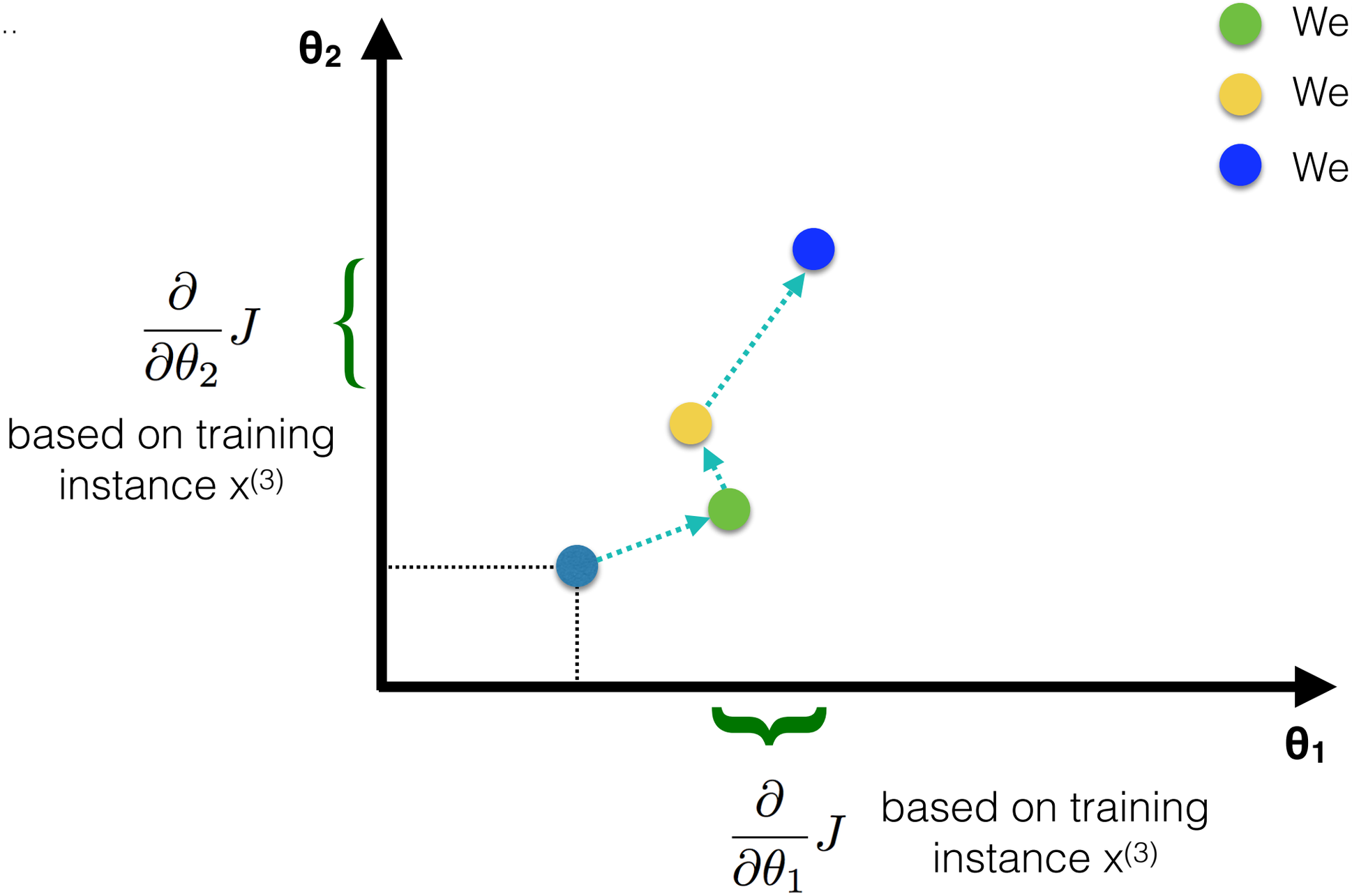
What if:

- Analyzes instance #1
- Computes gradient
- Updates weights

- Analyzes instance #2
- Computes gradient
- Updates weights

…

Stochastic Gradient Descent



- Initial weights
- Weights after $x^{(1)}$
- Weights after $x^{(2)}$
- Weights after $x^{(3)}$

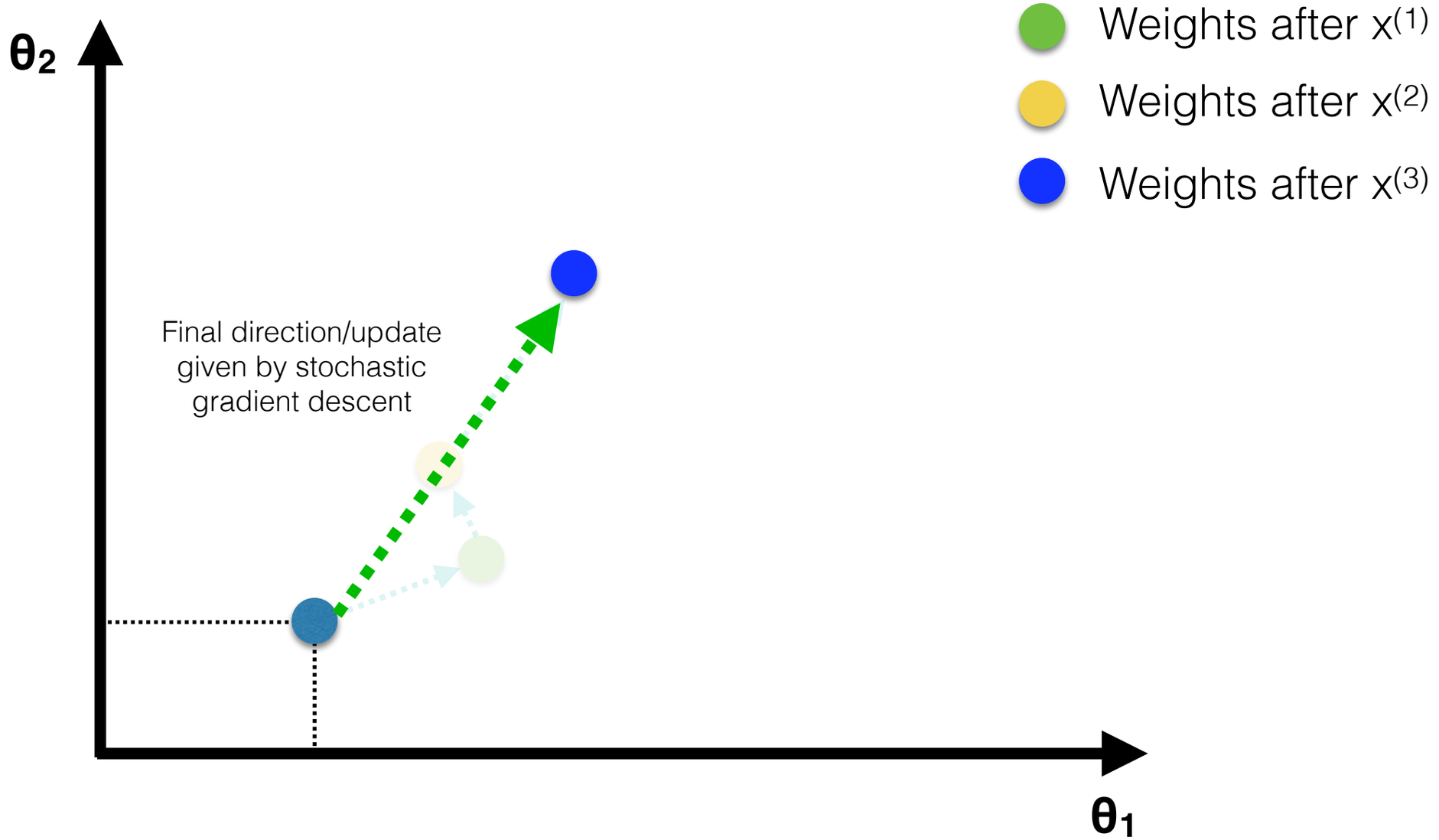
What if:

- Analyzes instance #1
- Computes gradient
- Updates weights

- Analyzes instance #2
- Computes gradient
- Updates weights

…

Stochastic Gradient Descent



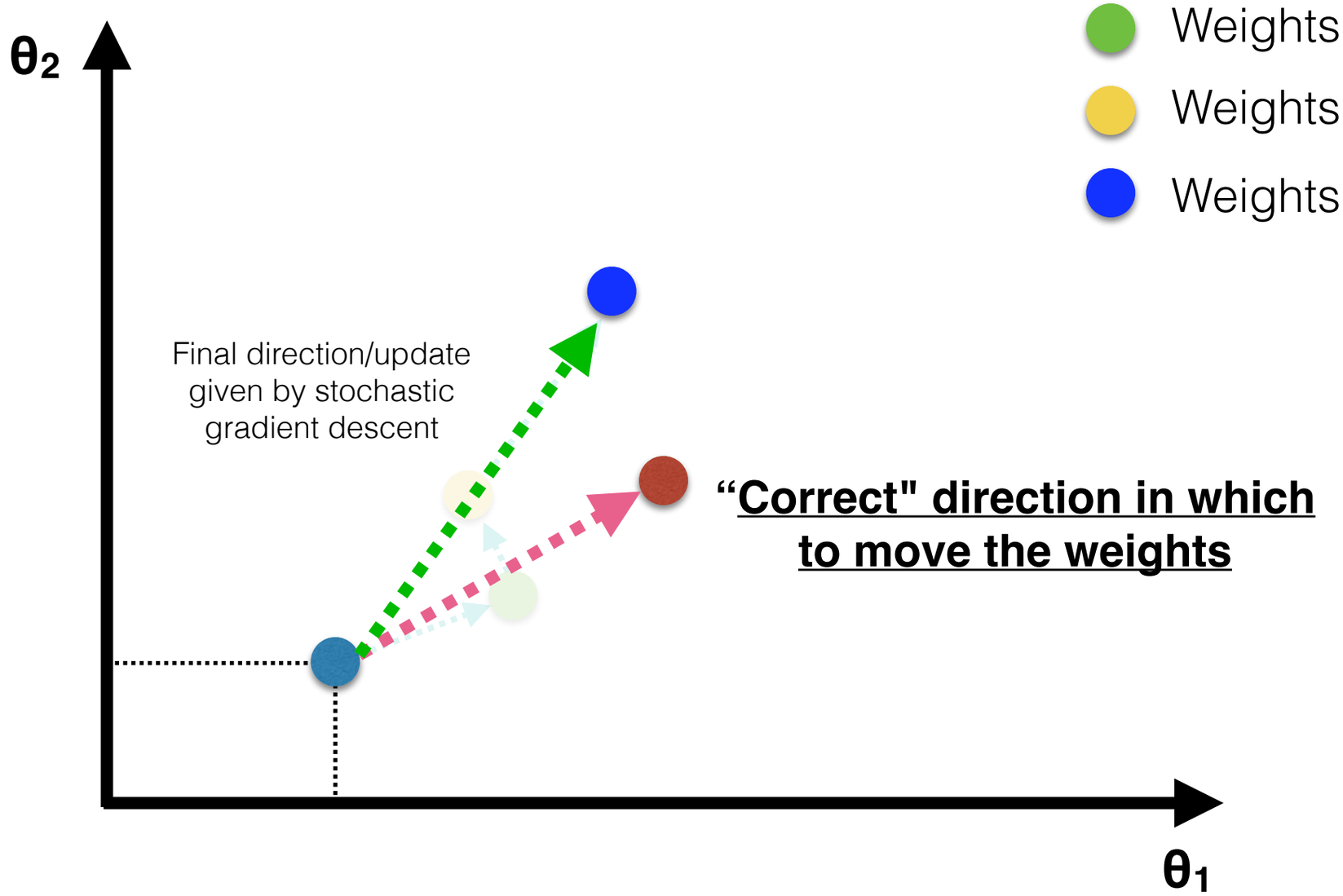
What if:

- Analyzes instance #1
- Computes gradient
- Updates weights

- Analyzes instance #2
- Computes gradient
- Updates weights

…

Stochastic Gradient Descent



What if:

- Analyzes instance #1
- Computes gradient
- Updates weights

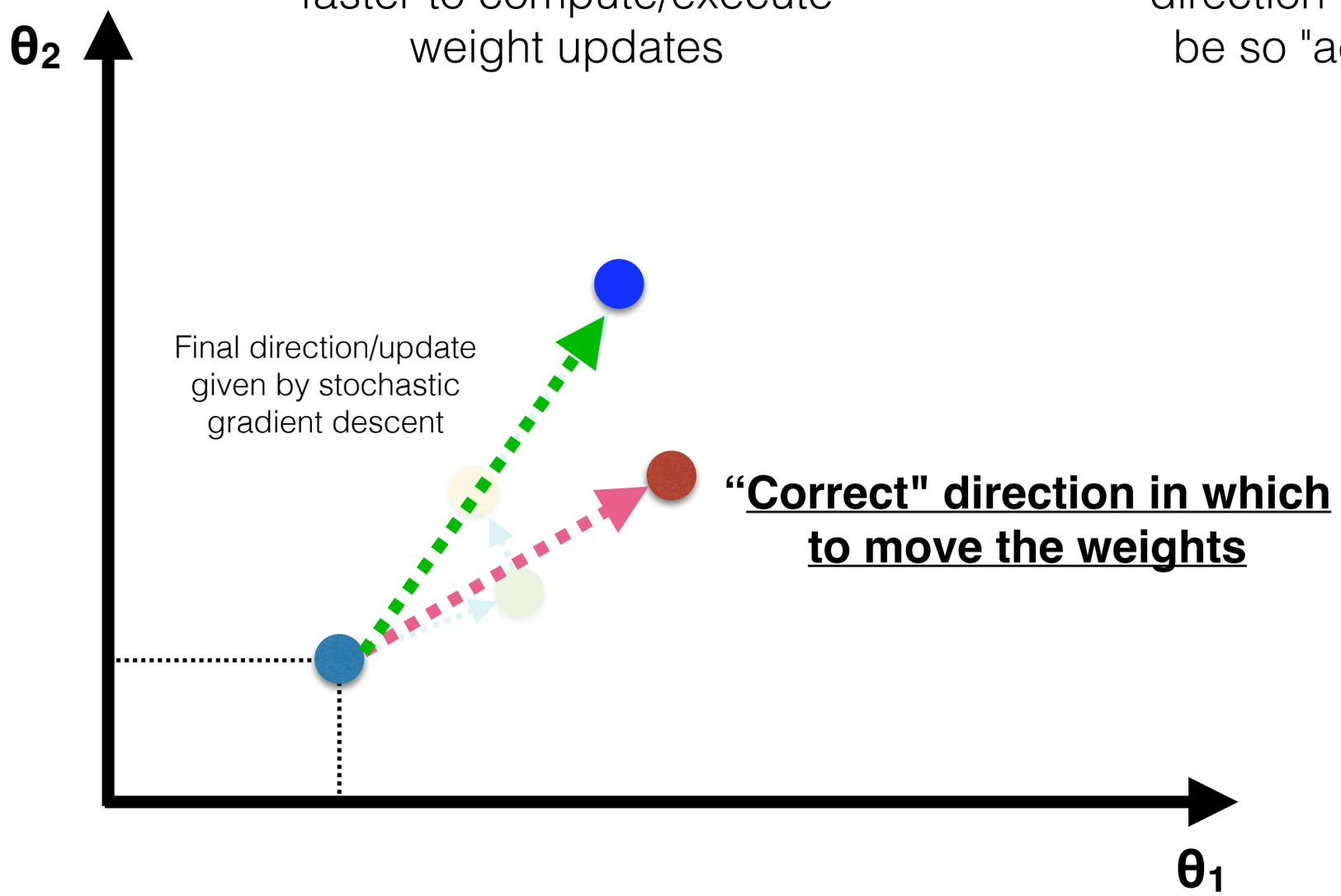
- Analyzes instance #2
- Computes gradient
- Updates weights

…

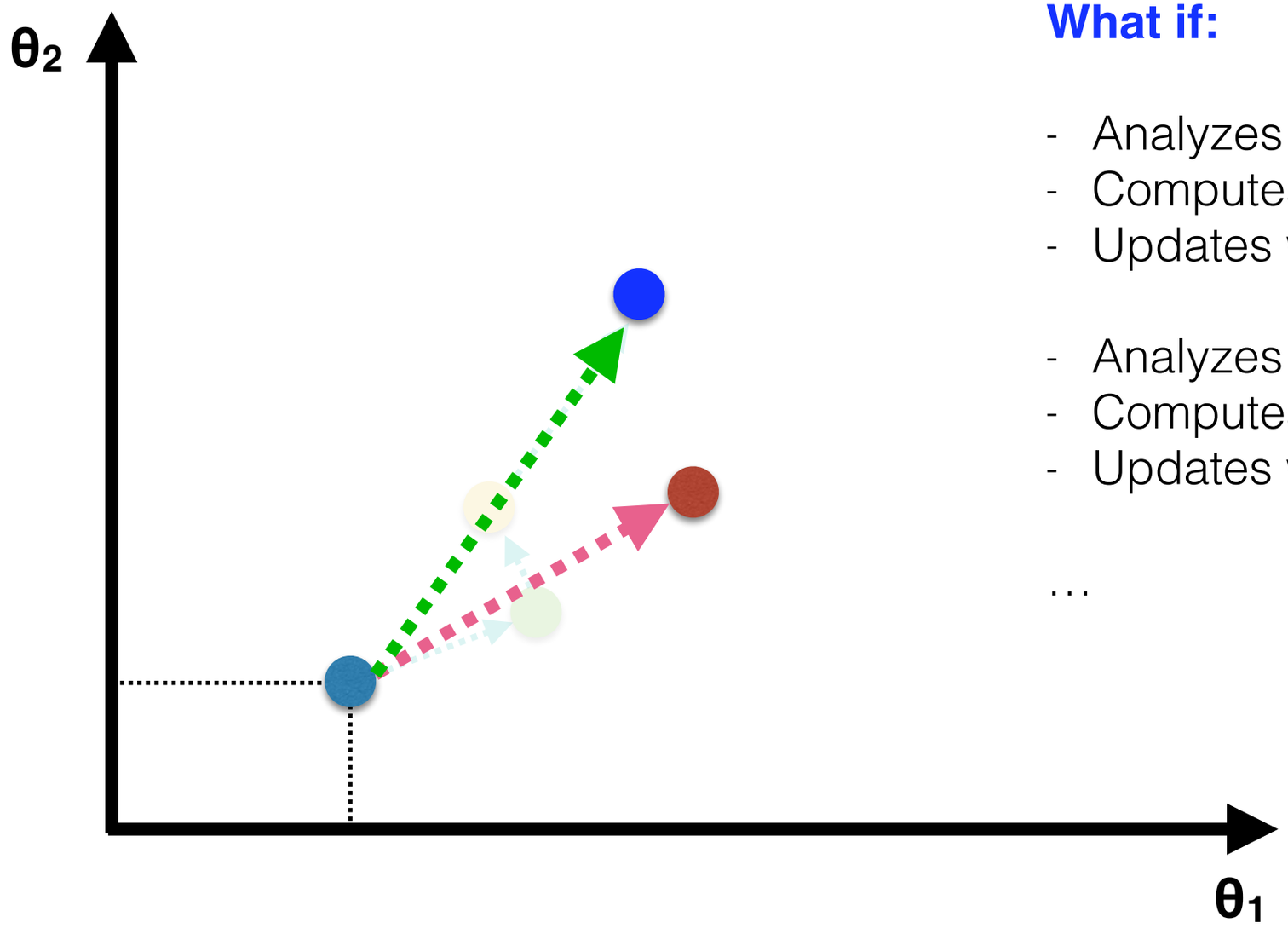
Stochastic Gradient Descent

Pros:
faster to compute/execute weight updates

Cons: the final resulting direction might not be so "accurate"



Mini-batch Gradient Descent



What if:

- Analyzes instances **#1..#50**
- Computes gradient
- Updates weights

- Analyzes instances **#2..#100**
- Computes gradient
- Updates weights

…

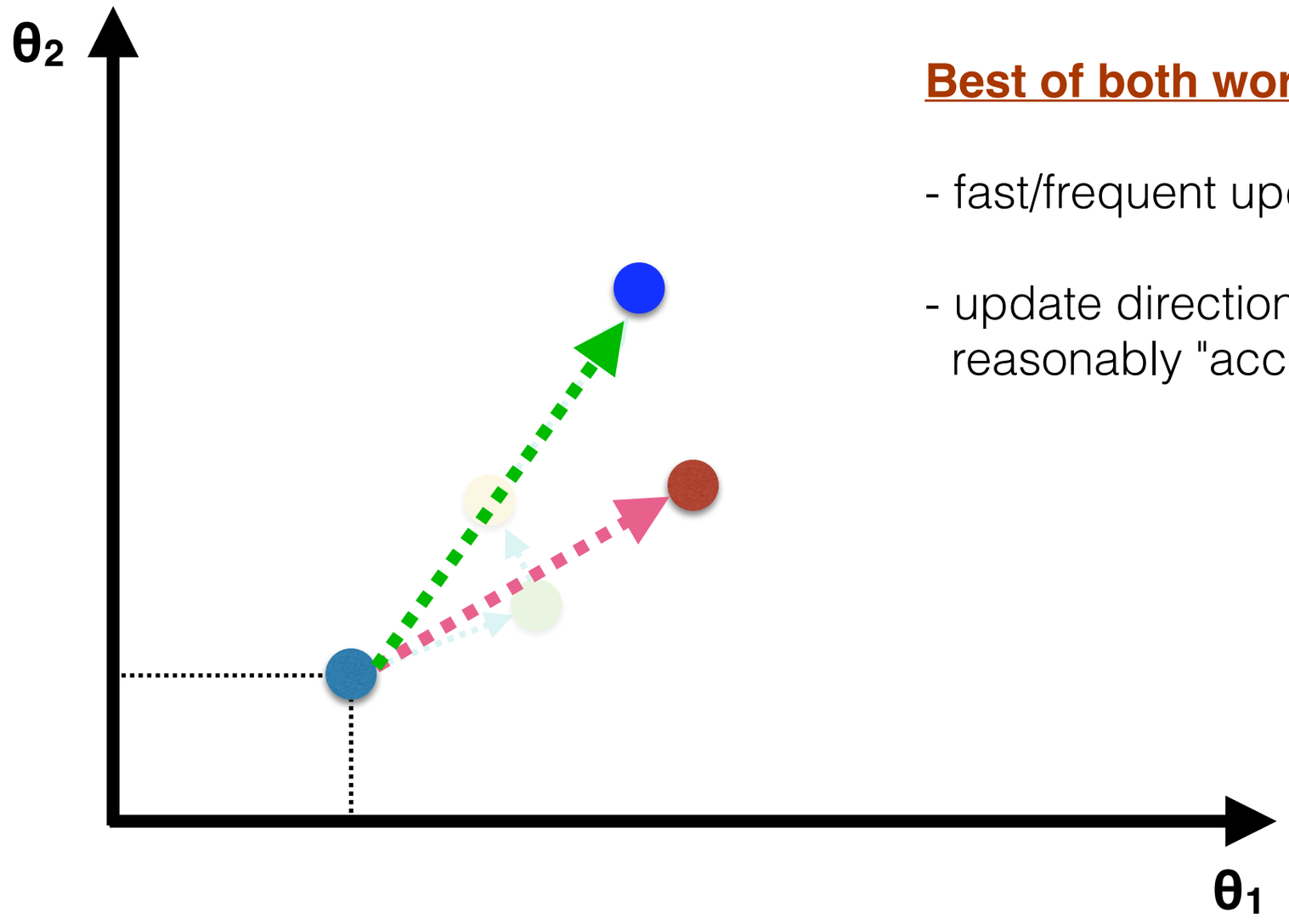
Mini-batch Gradient Descent

What if:

- Analyzes instances **#1..#50**
- Computes gradient
- Updates weights

- Analyzes instances **#2..#100**
- Computes gradient
- Updates weights

…



Best of both worlds:

- fast/frequent updates to weights

- update directions are reasonably "accurate"

Batch Gradient Descent

- Repeat until convergence:
  - Analyzes instance **1**
  - Computes/accumulates gradients
  - Analyzes instance **2**
  - Computes/accumulates gradients
  - Analyzes instance **n**
  - Computes/accumulates gradients
  - Uses total accumulated gradients to updated the weights

Equivalent to mini-batch with K=n

Stochastic Gradient Descent

- Repeat until convergence:
  - Analyzes instance **1**
  - Computes gradients
  - Updates weights
  - Analyzes instance **2**
  - Computes gradients
  - Updates weights
  - …
  - Analyzes instance **n**
  - Computes gradients
  - Updates weights

Equivalent to mini-batch with K=1

Mini-batch Gradient Descent

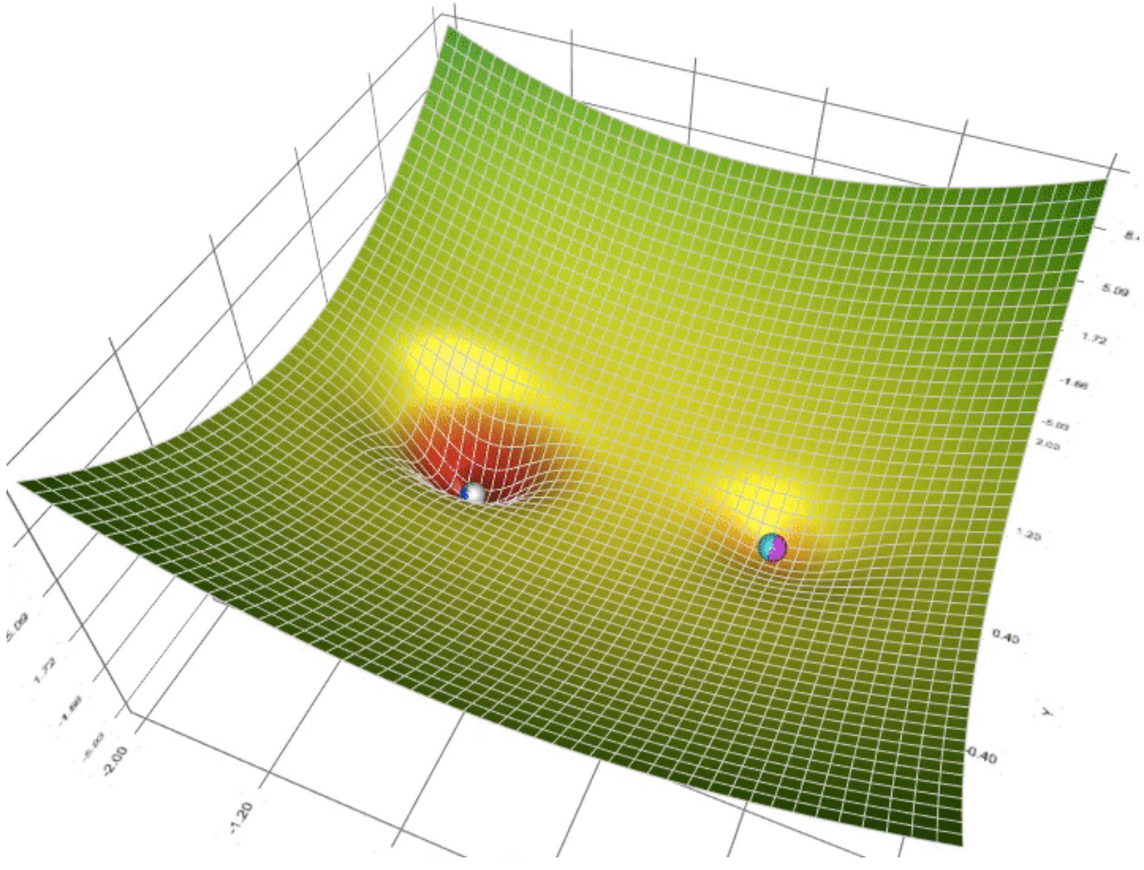
- Repeat until convergence:
  - Analyzes instances **1..K**
  - Computes gradients
  - Updates weights
  - Analyzes instances **K+1..2K**
  - Computes gradients
  - Updates weights
  - …
  - Analyzes instances **2K+1..3K**
  - Computes gradients
  - Updates weights

Chooses:
- **mini-batch size (K)**
  or
- **number of mini-batches**

Heuristics to Accelerate Convergence

- **Standard algorithm uses only the current gradient to update the weights**

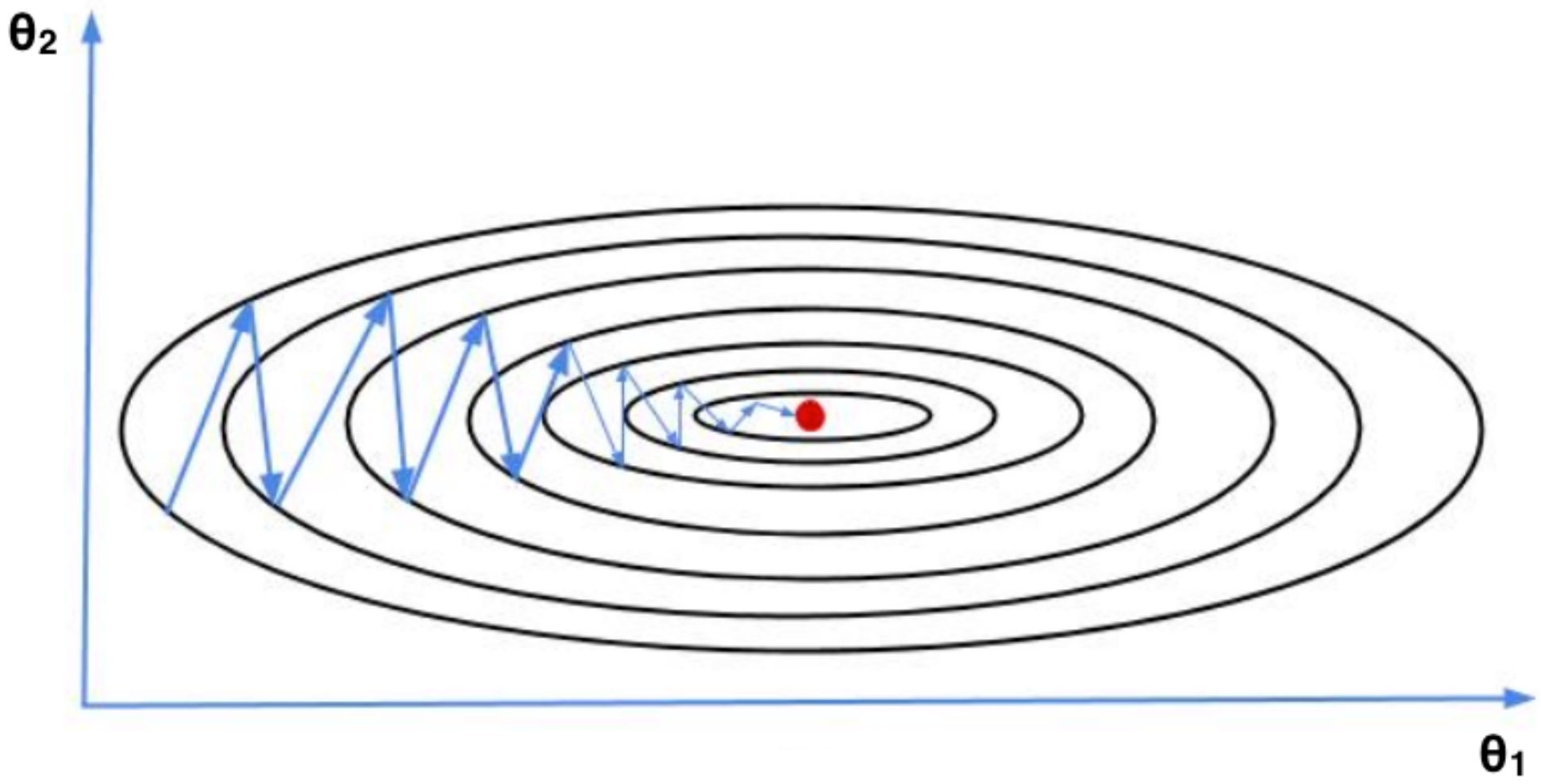
$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$



Heuristics to Accelerate Convergence

- **Standard algorithm uses only the current gradient to update the weights**

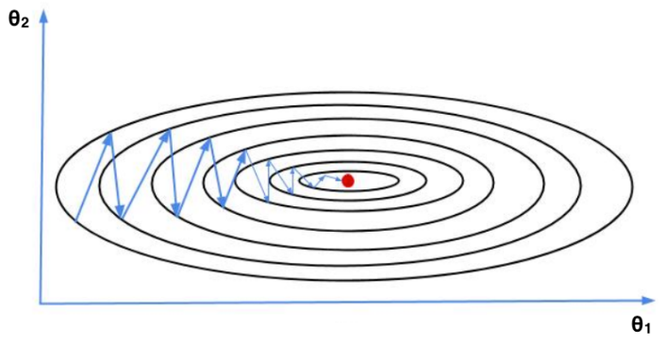
$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$



Heuristics to Accelerate Convergence

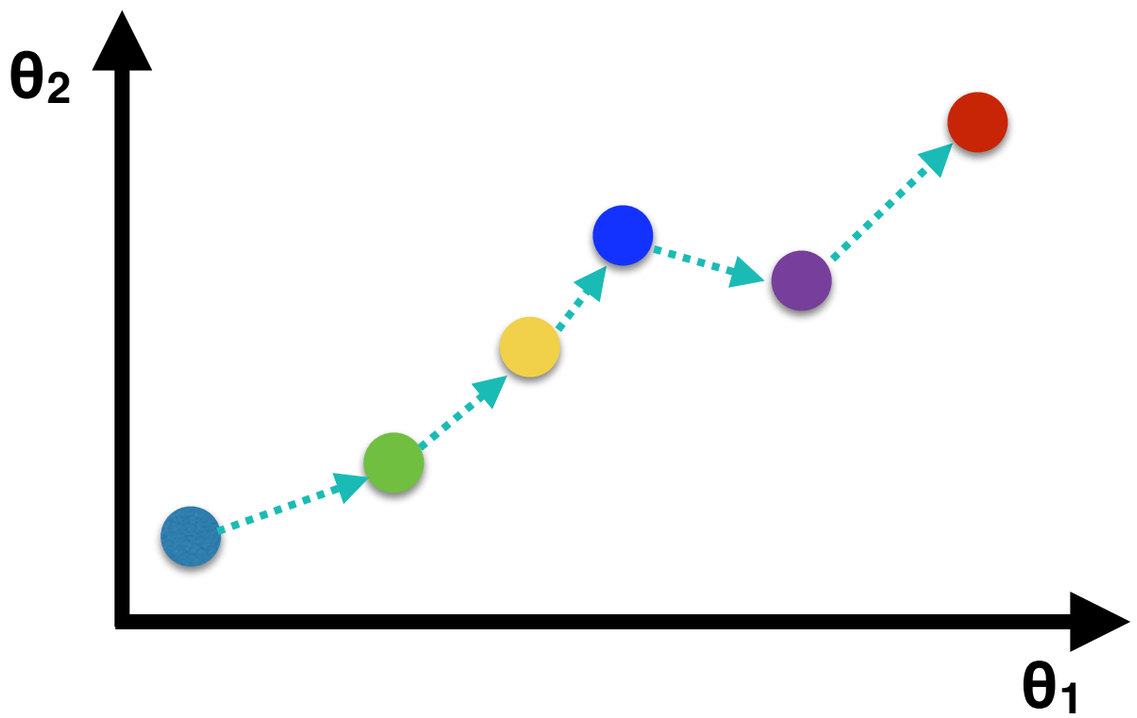
- **Standard algorithm uses only the <u>current gradient</u> to update the weights**

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$



- Gradient points in the <u>current</u> direction we should follow to decrease error J
- But might oscillate and we might have to “correct”/undo these updates <u>later on</u>

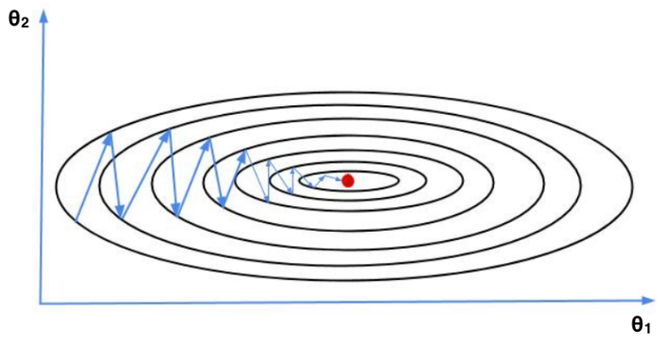
- Idea
  - Instead of taking only the current gradient into account when updating weights
  - Takes into account also previous updated directions



Heuristics to Accelerate Convergence

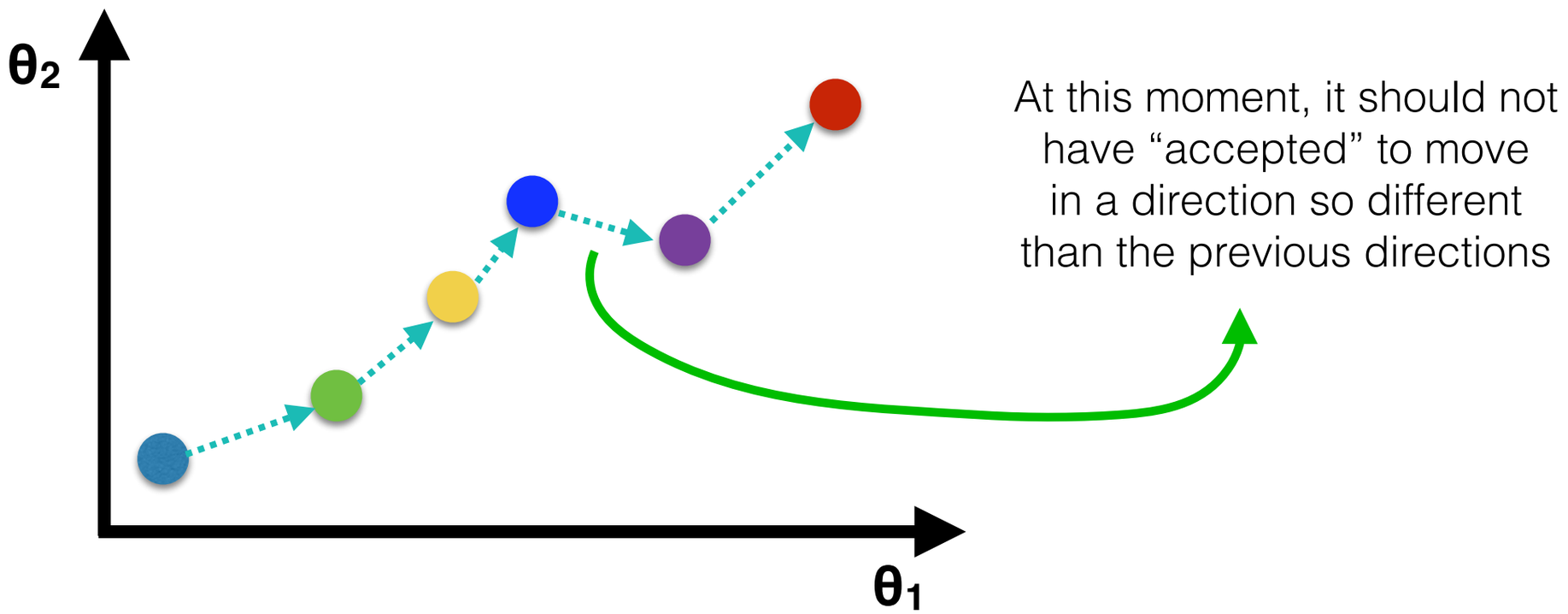
- **Standard algorithm uses only the <u>current gradient</u> to update the weights**

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$



- Gradient points in the <u>current</u> direction we should follow to decrease error J
- But might oscillate and we might have to "correct"/undo these updates <u>later on</u>

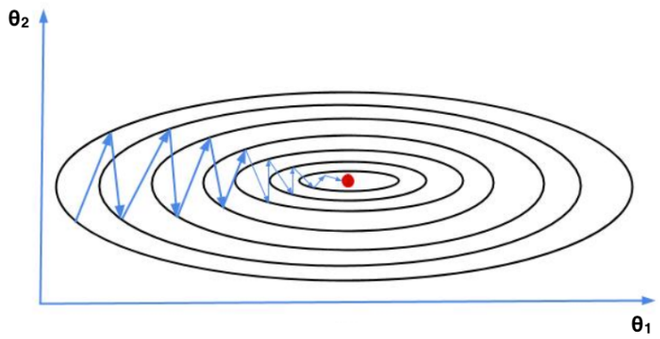
- Idea
  - Instead of taking only the current gradient into account when updating weights
  - Takes into account also previous updated directions



Heuristics to Accelerate Convergence

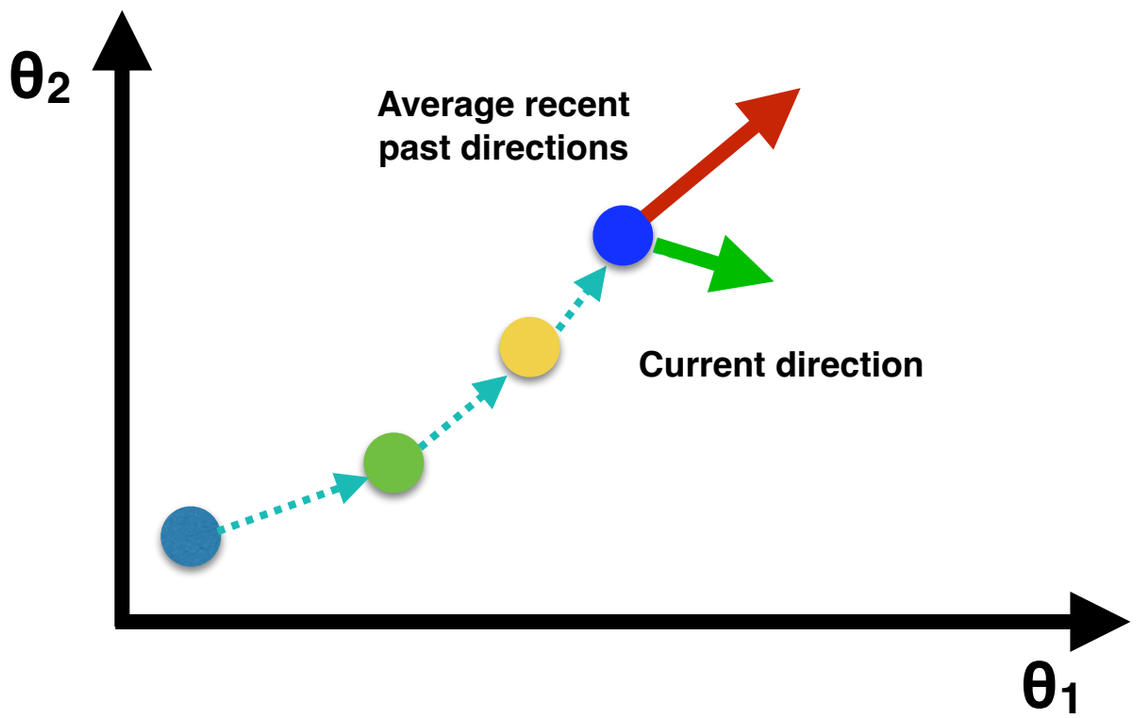
- **Standard algorithm uses only the <u>current gradient</u> to update the weights**

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$



- Gradient points in the <u>current</u> direction we should follow to decrease error J
- But might oscillate and we might have to “correct”/undo these updates <u>later on</u>

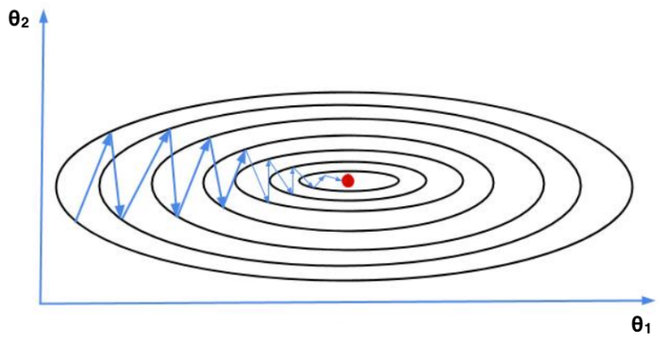
- Idea
  - Instead of taking only the current gradient into account when updating weights
  - Takes into account also previous updated directions



Heuristics to Accelerate Convergence

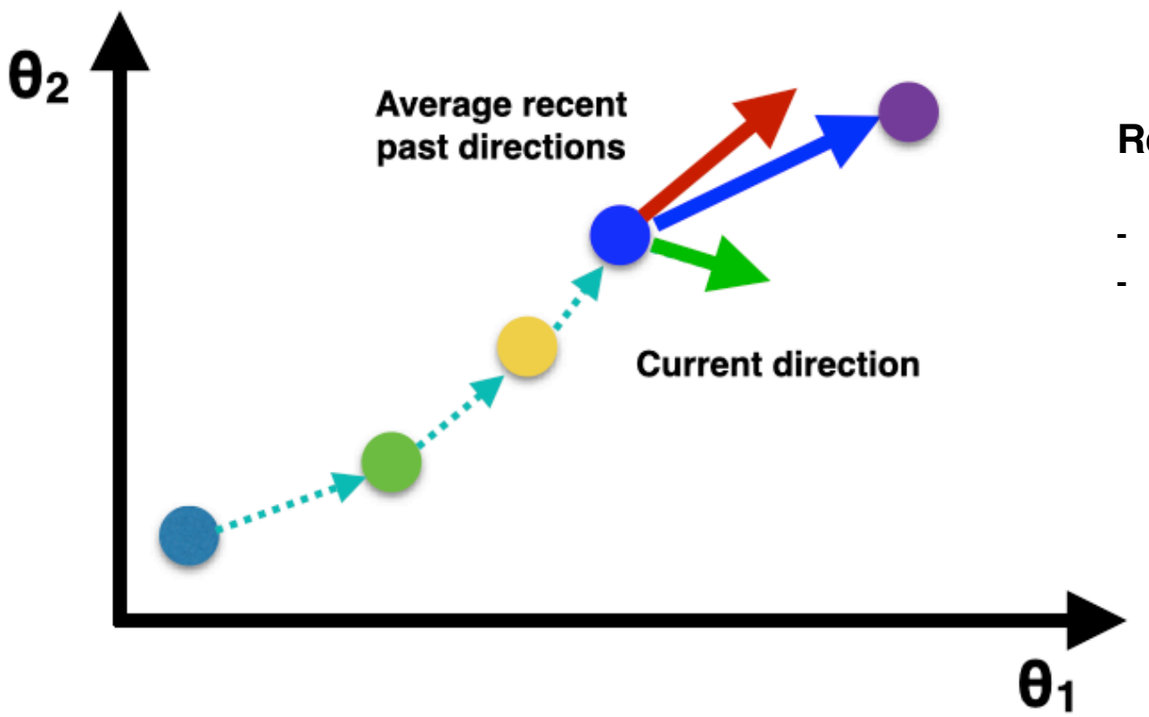
- **Standard algorithm uses only the <u>current gradient</u> to update the weights**

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$



- Gradient points in the <u>current</u> direction we should follow to decrease error J
- But might oscillate and we might have to “correct”/undo these updates <u>later on</u>

- Idea
  - Instead of taking only the current gradient into account when updating weights
  - Takes into account also previous updated directions



Resulting direction: something like

- **90% of average recent past directions**
- **10% of current direction**

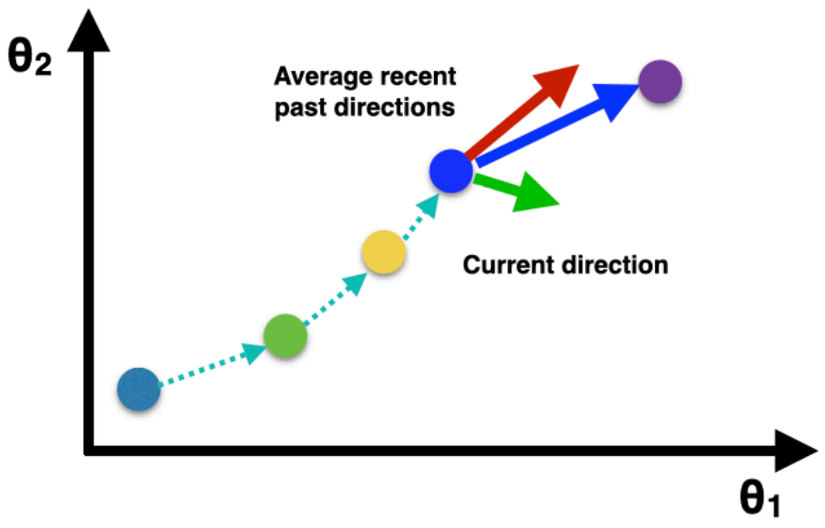
Gradient Descent with Momentum

Heuristics to Accelerate Convergence

Standard/classic update

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha \frac{\partial}{\partial \theta_{ij}^{l}} J$$

Gradient Descent with Momentum



Resulting direction: something like

- **90% of average recent past directions**
- **10% of current direction**

e.g., 0.9 (90%)

$$z_{ij}^{l} := \beta\, z_{ij}^{l} + \frac{\partial}{\partial \theta_{ij}^{l}} J$$

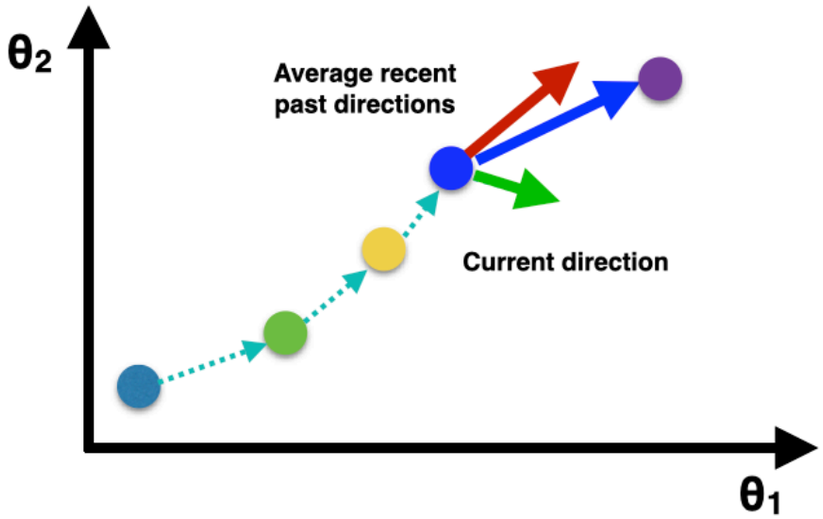
// direction ***z***: combines *previous*
// average direction with *current* direction

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha z_{ij}^{l}$$

// updates weights in a direction that
// takes previous directions into account

Heuristics to Accelerate Convergence

Gradient Descent with Momentum



$$z_{ij}^{l} := \beta\, z_{ij}^{l} + \frac{\partial}{\partial \theta_{ij}^{l}} J$$

// direction ***z***: combines *previous*
// average direction with *current* direction

$$\theta_{ij}^{l} := \theta_{ij}^{l} - \alpha z_{ij}^{l}$$

// updates weights in a direction that
// takes previous directions into account

Demo

https://distill.pub/2017/momentum/

Heuristics to Accelerate Convergence

Gradient Descent with Momentum

- This is a *heuristic* to try to accelerate convergence of gradient descent algorithms
- In practice, it often accelerates convergence by more than 10x
  - (in terms of number of updates required to converge)
- Time it takes to converge becomes less sensitive to the step size α

- Widely used by gradient descent methods
  - Not only neural networks
  - Linear regression, logistic regression, etc

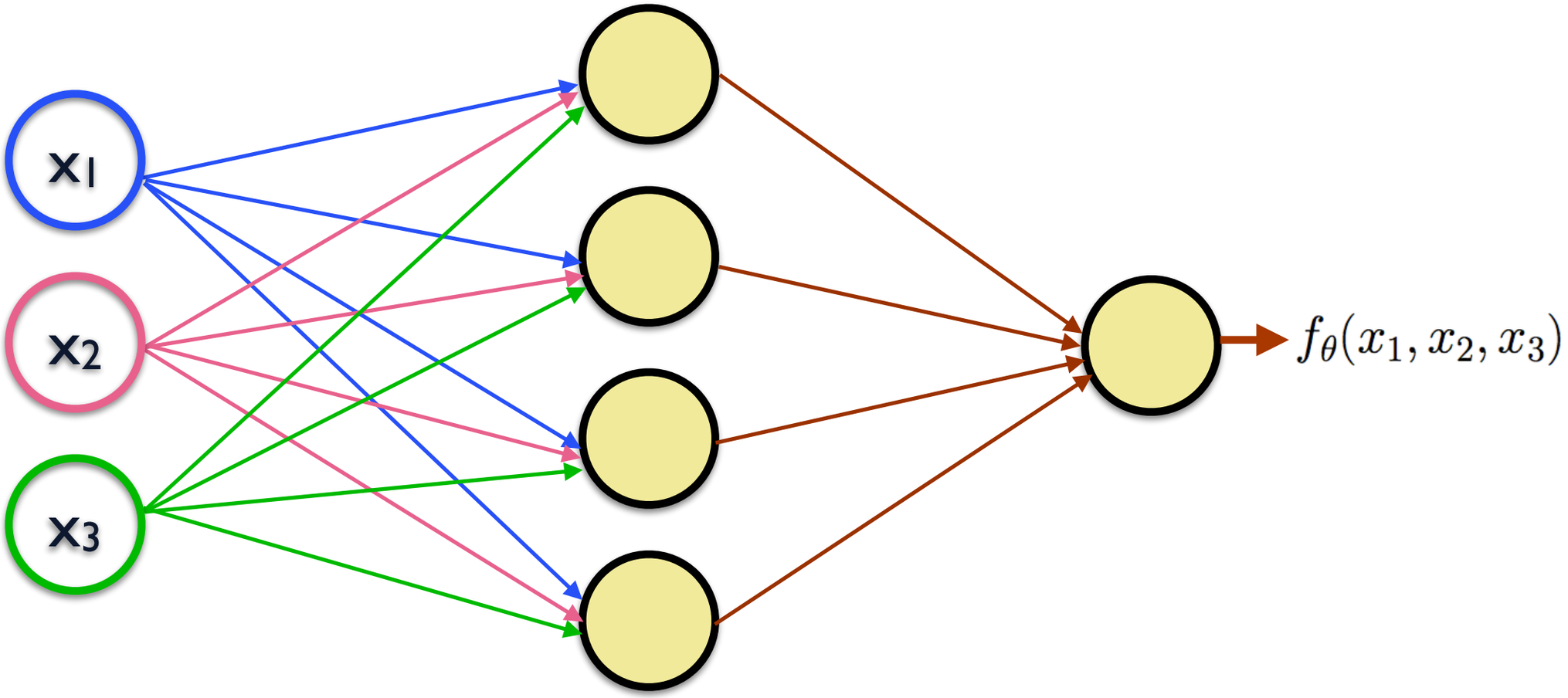
- Many other heuristics are often used:
  - Momentum, RMS propagation, ADAGRAD, ADADELTA, Adam, etc.

- They just require a way of computing the current gradient (e.g., via backpropagation)
  - and then, these heuristics use and combine gradient directions more “intelligently"

Convolutional Neural Networks (CNNs)

https://www.youtube.com/watch?v=n5uP_LP9SmM

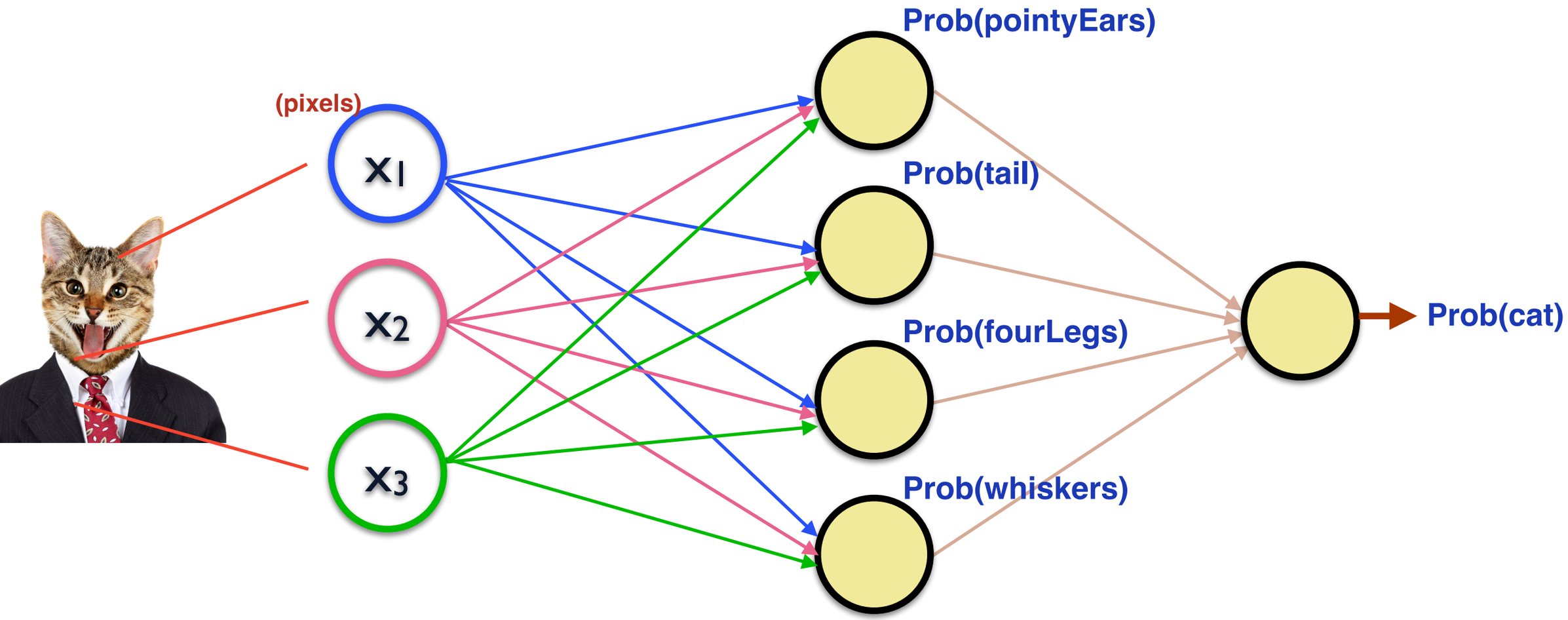
Neural Networks



Network (graph) of connected neurons

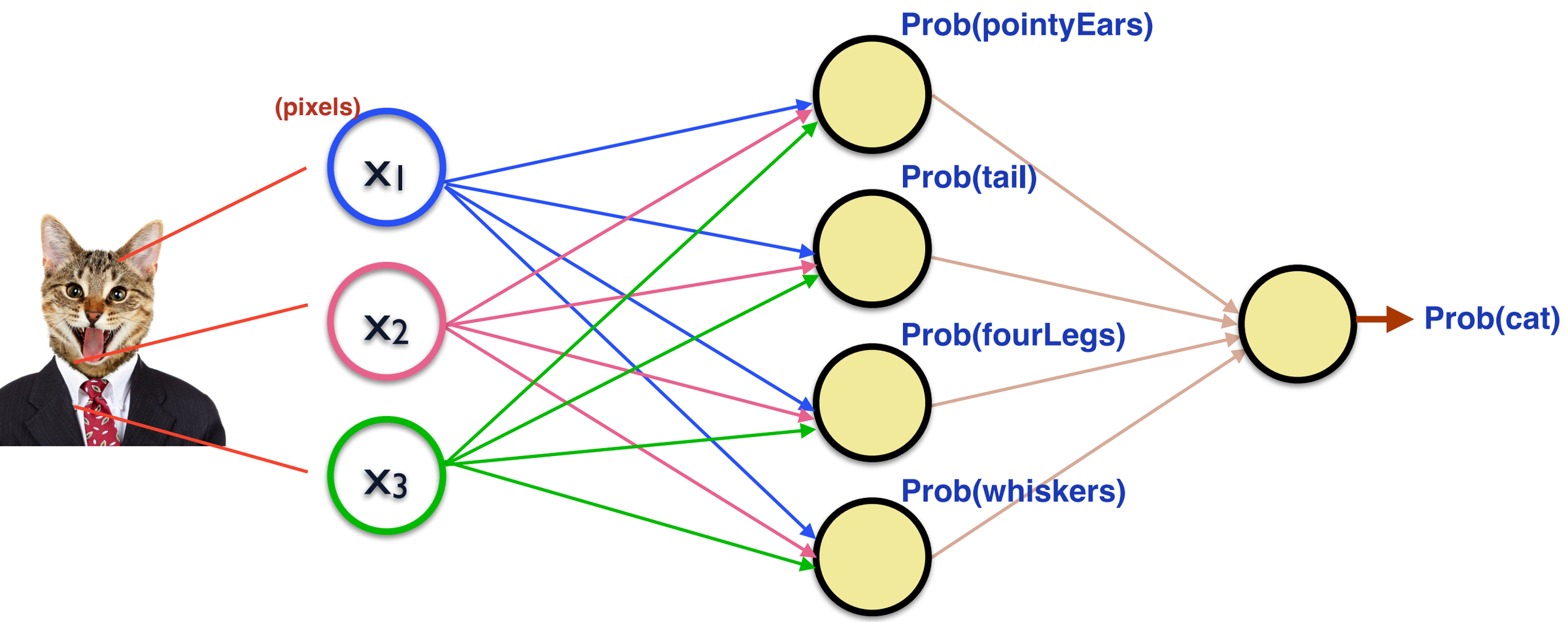
Neural Networks

By training the network, hidden neurons automatically learn *features* that are important for the classification task



Why is it advantageous to combine neurons?

Convolutional Neural Networks

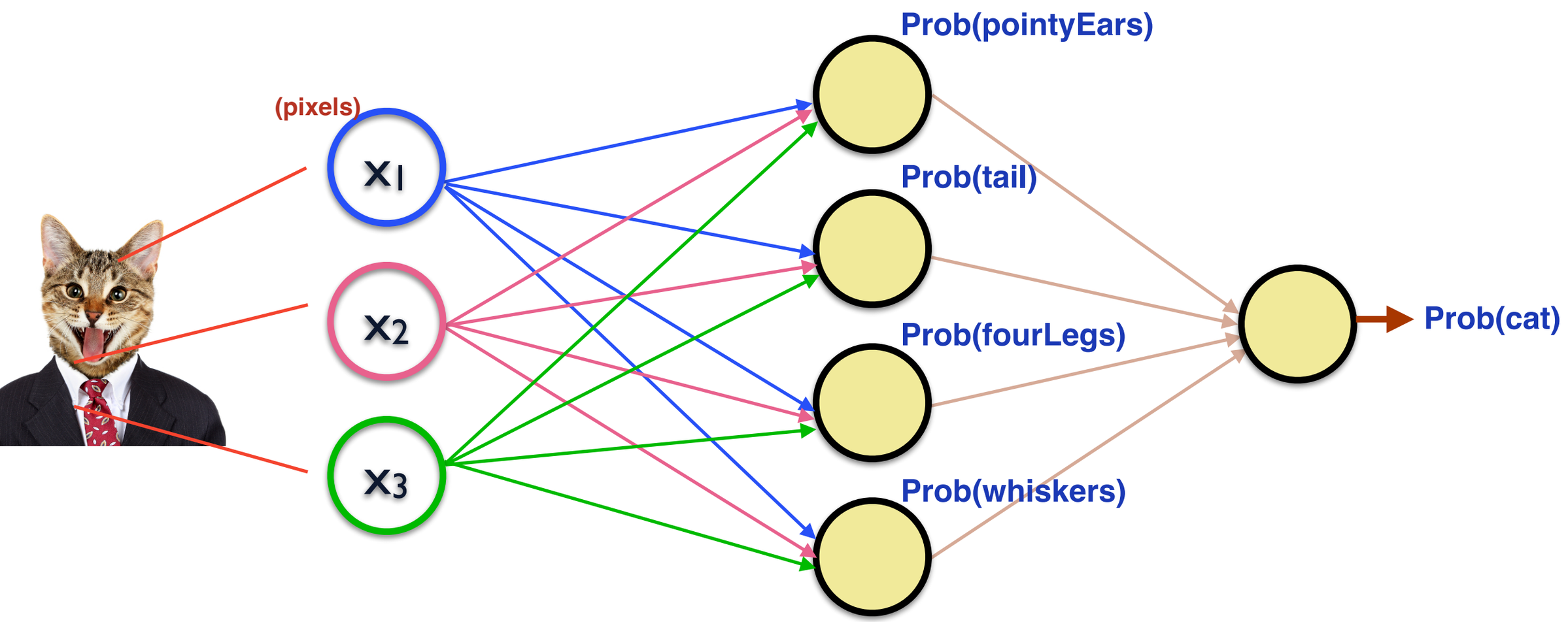


Assume that input images are 28x28 pixels (i.e., 784 inputs are given to the network)
Assume 100 neurons in the hidden layer—each one connected to all inputs/pixels
→ 78k weights just in the *first* layer of the network!

Convolutional Neural Networks

smaller number of weights, compared to traditional neural networks, e.g., if applied to image classification

Convolutional Neural Networks

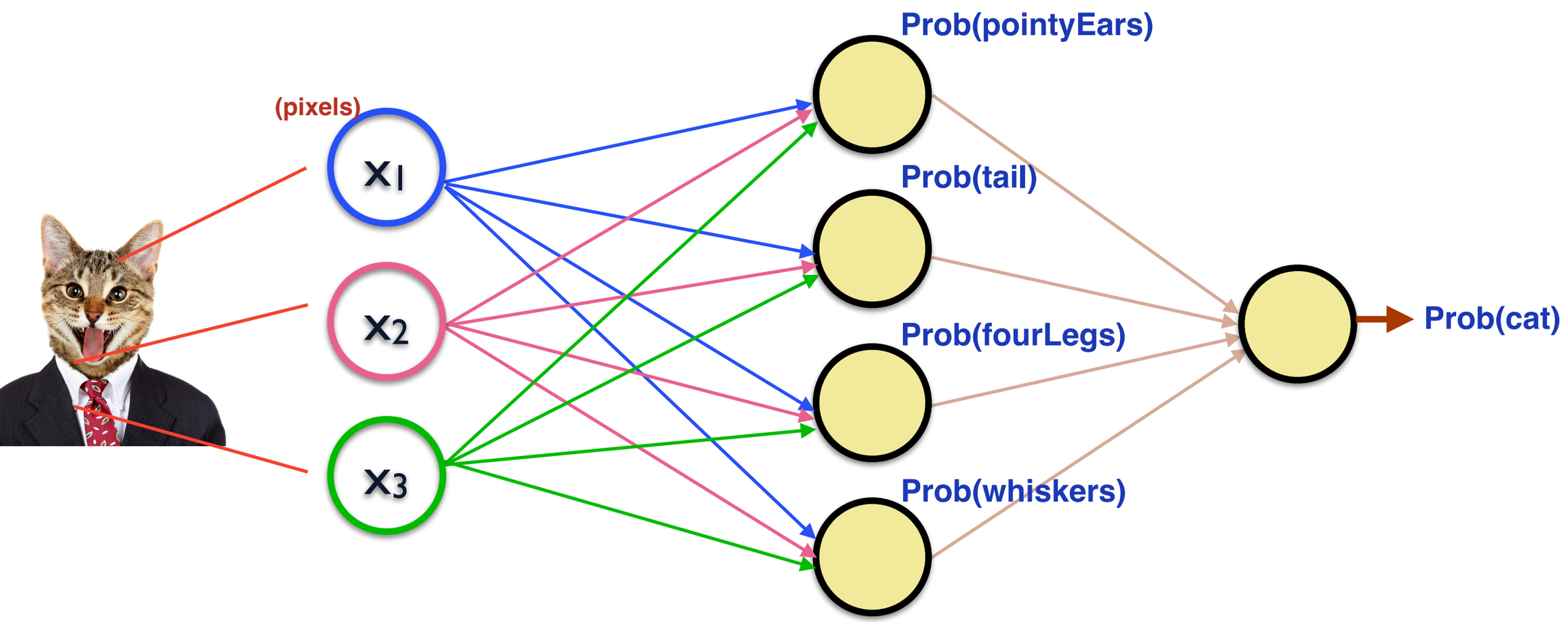


Assume that input images are 28x28 pixels (i.e., 784 inputs are given to the network)
Assume 100 neurons in the hidden layer—each one connected to all inputs/pixels
→ 78k weights just in the *first* layer of the network!

Convolutional Neural Networks

more easily learn the above-mentioned "detectors" of features
→ convolution filters applied to image to detect the presence of a given feature or characteristic

Convolutional Neural Networks



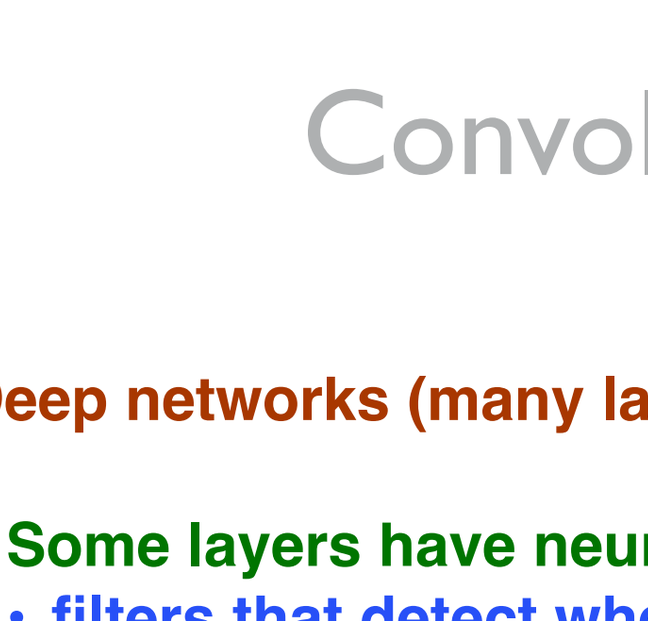
Assume that input images are 28x28 pixels (i.e., 784 inputs are given to the network)
Assume 100 neurons in the hidden layer—each one connected to all inputs/pixels
→ 78k weights just in the *first* layer of the network!

Convolutional Networks

better than traditional networks when applied to image classification

Convolutional Neural Networks

- **Deep networks (many layers)**
  - **Some layers have neurons that implement a *convolution***
    - **filters that detect whether feature/characteristic exists in a given region**



filter to detect vertical edges

pixels are "*on*" in regions containing vertical edges, "*off*" otherwise

filter to detect horizontal edges

pixels are "*on*" in regions containing horizontal edges, "*off*" otherwise

Convolutional Neural Networks

- **Deep networks (many layers)**
  - **Some layers have neurons that implement a *convolution***
    - **filters that detect whether feature/characteristic exists in a given region**

filter to
detect eyes

pixels are "*on*"
in regions containing
eyes,
"*off*"otherwise

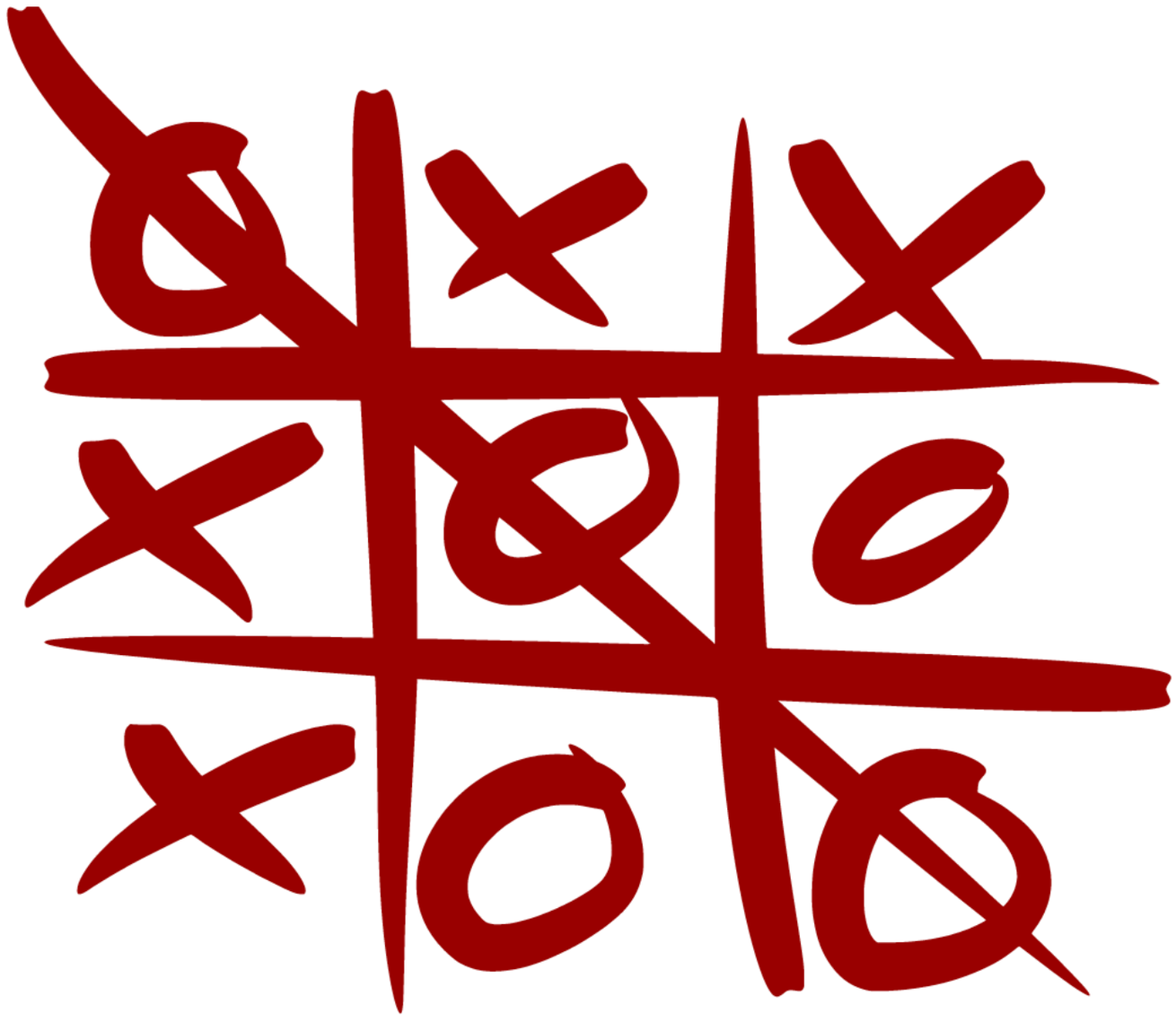
Convolutional Neural Networks

- **Deep networks (many layers)**
  - **Some layers have neurons that implement a *convolution***
    - **filters that detect whether feature/characteristic exists in a given region**

- **Which filters should be used to help classifying input images?**
  - **eye detectors**
  - **pointy ear detectors**
  - **horizontal edge detectors**
  - **etc**

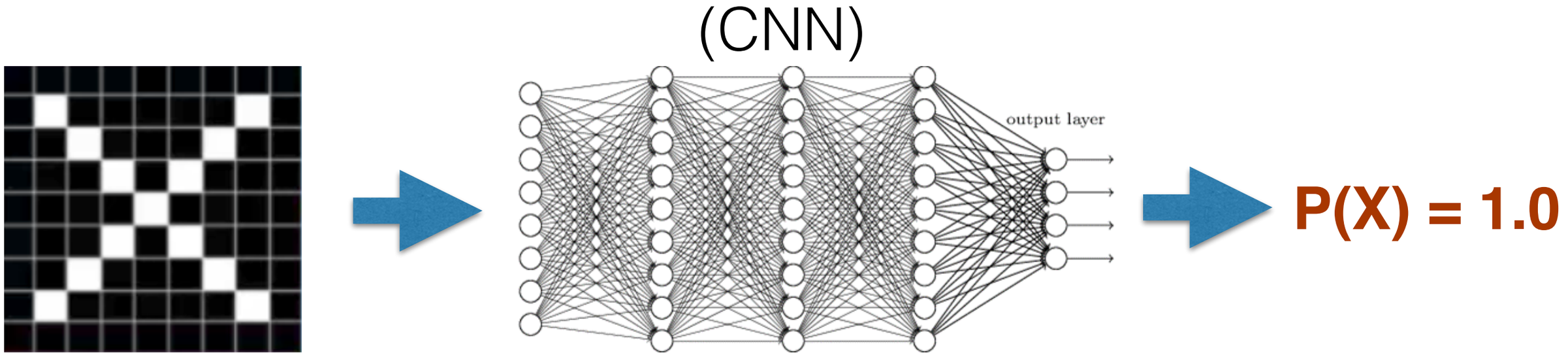
- **Convolutional networks *learn* filters to detect features useful for classification**

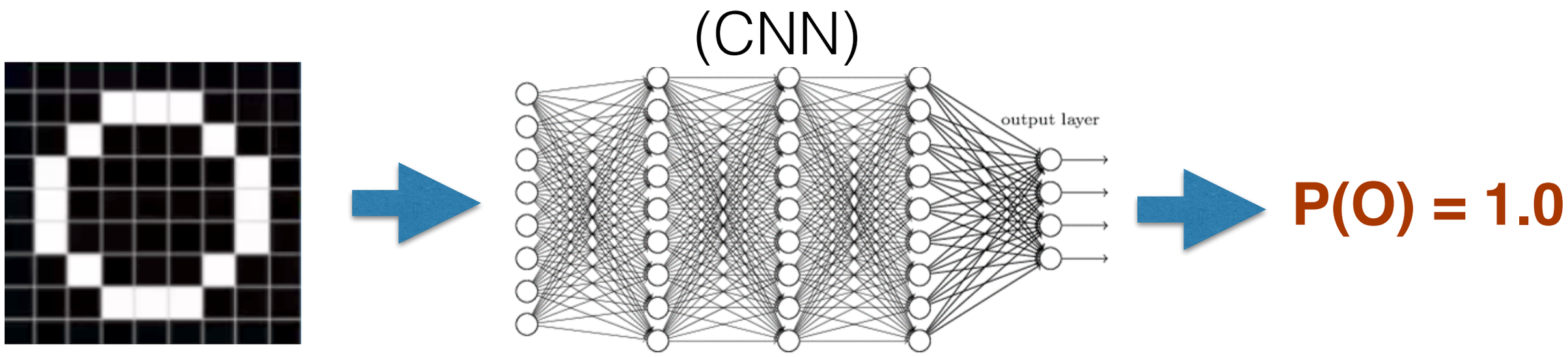
Convolutional Neural Networks



Network capable of classifying images as drawings of an “X" or an "O"

Convolutional Neural Networks

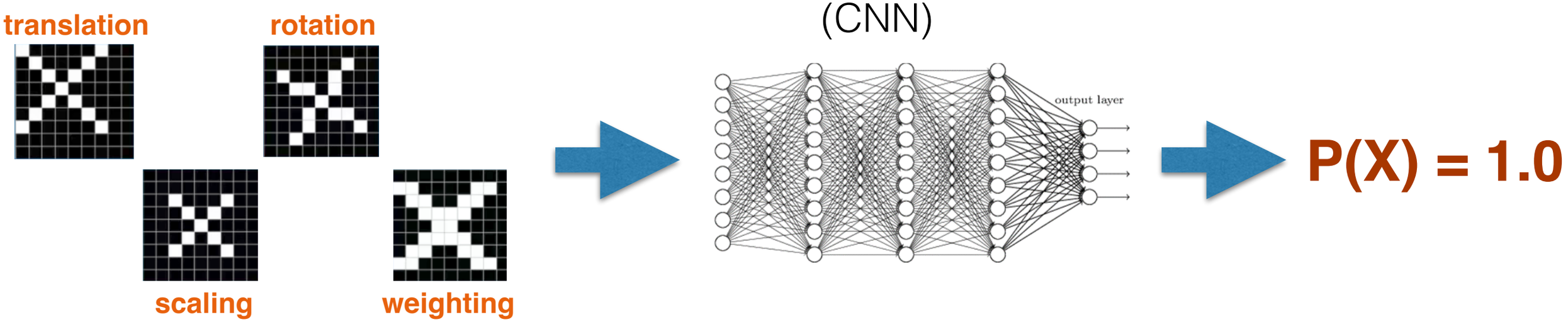


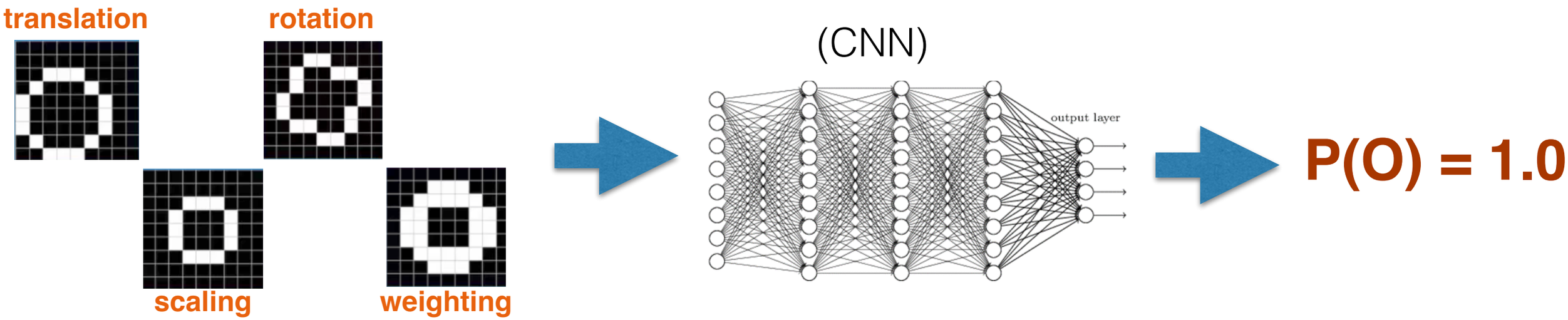


Network capable of classifying images as drawings of an “X" or an "O"

Source: How do Convolutional Neural Networks work? (Brandon Rohrer 2016)

Convolutional Neural Networks

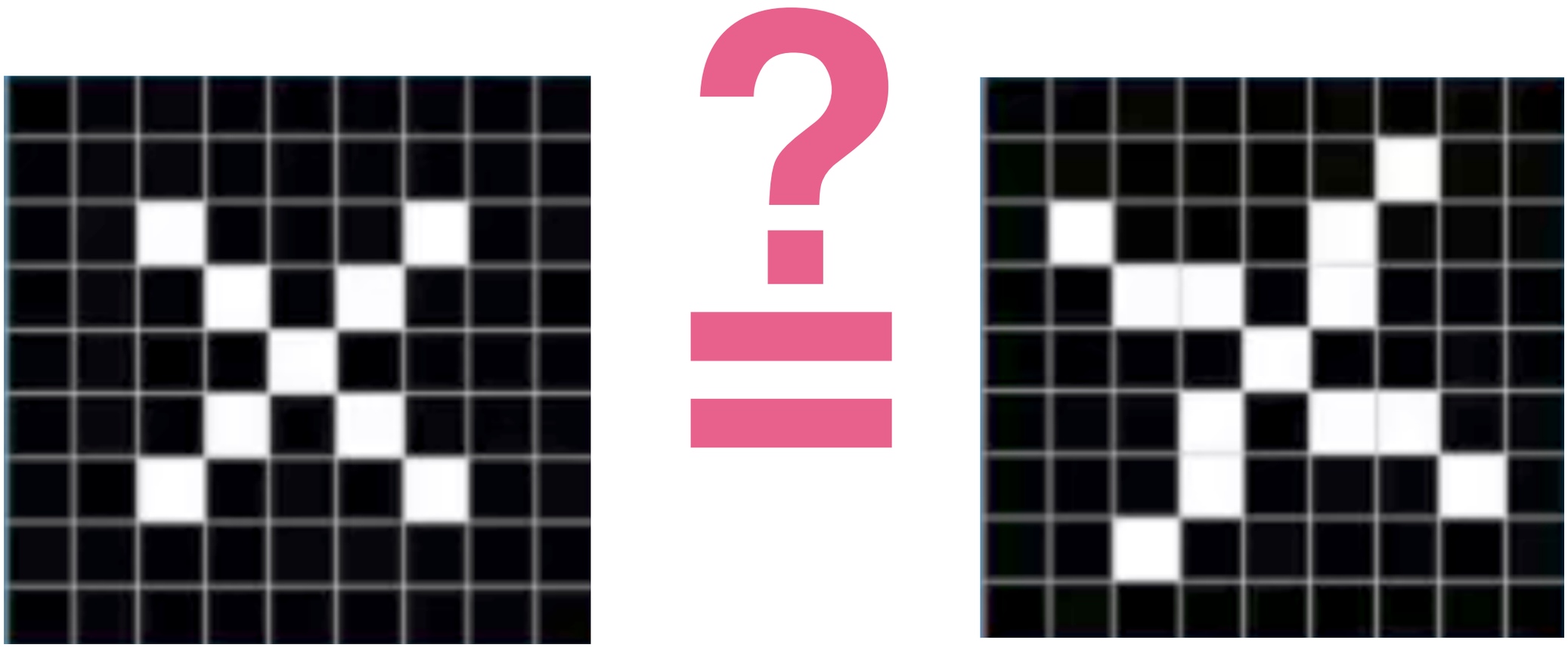




Network capable of classifying images as drawings of an “X" or an "O"

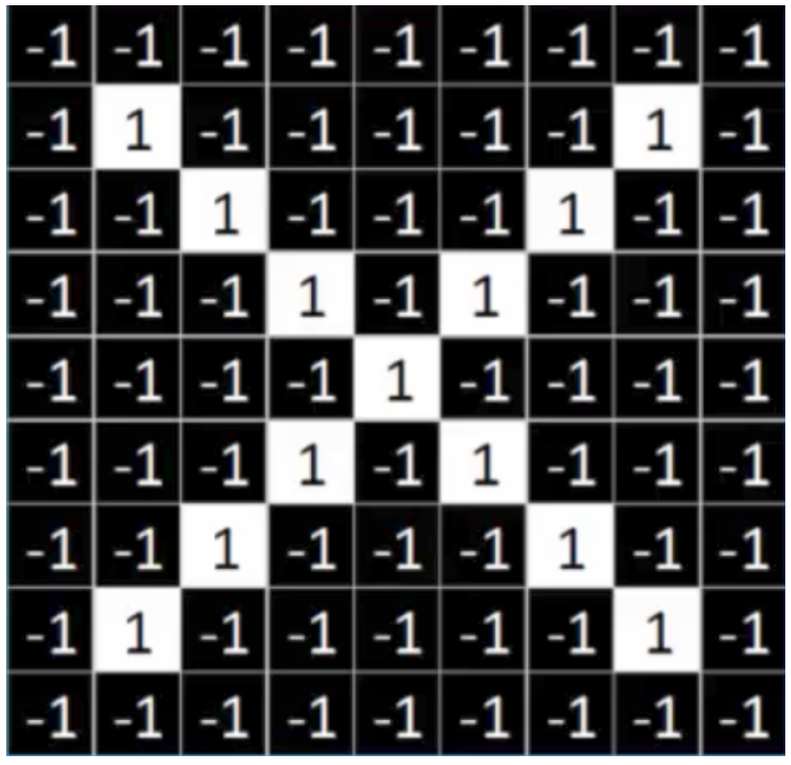
Should work even if images are translated, rotated, etc!

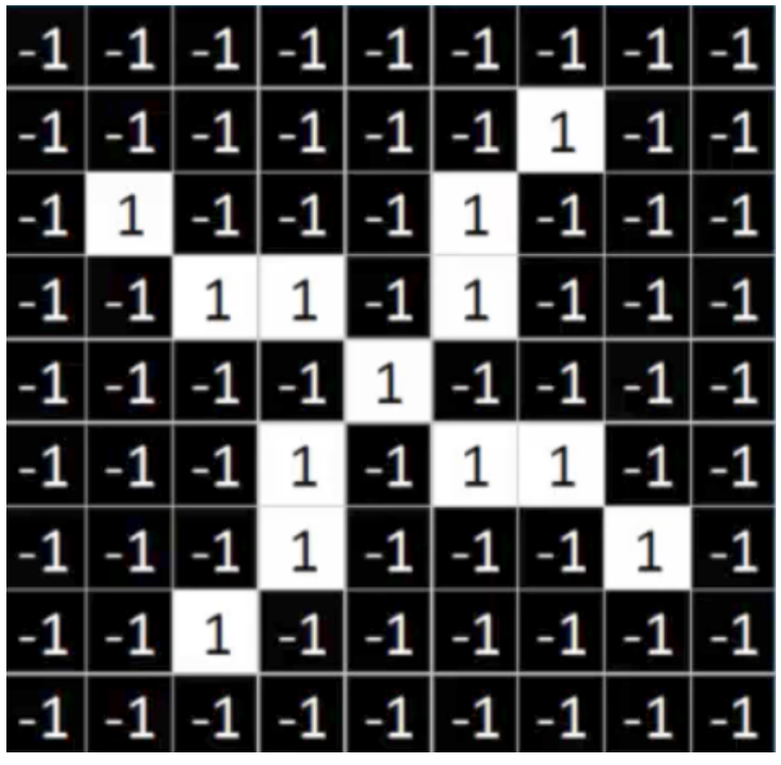
Convolutional Neural Networks



Why is it hard for the network to know whether these images belong to the same class?

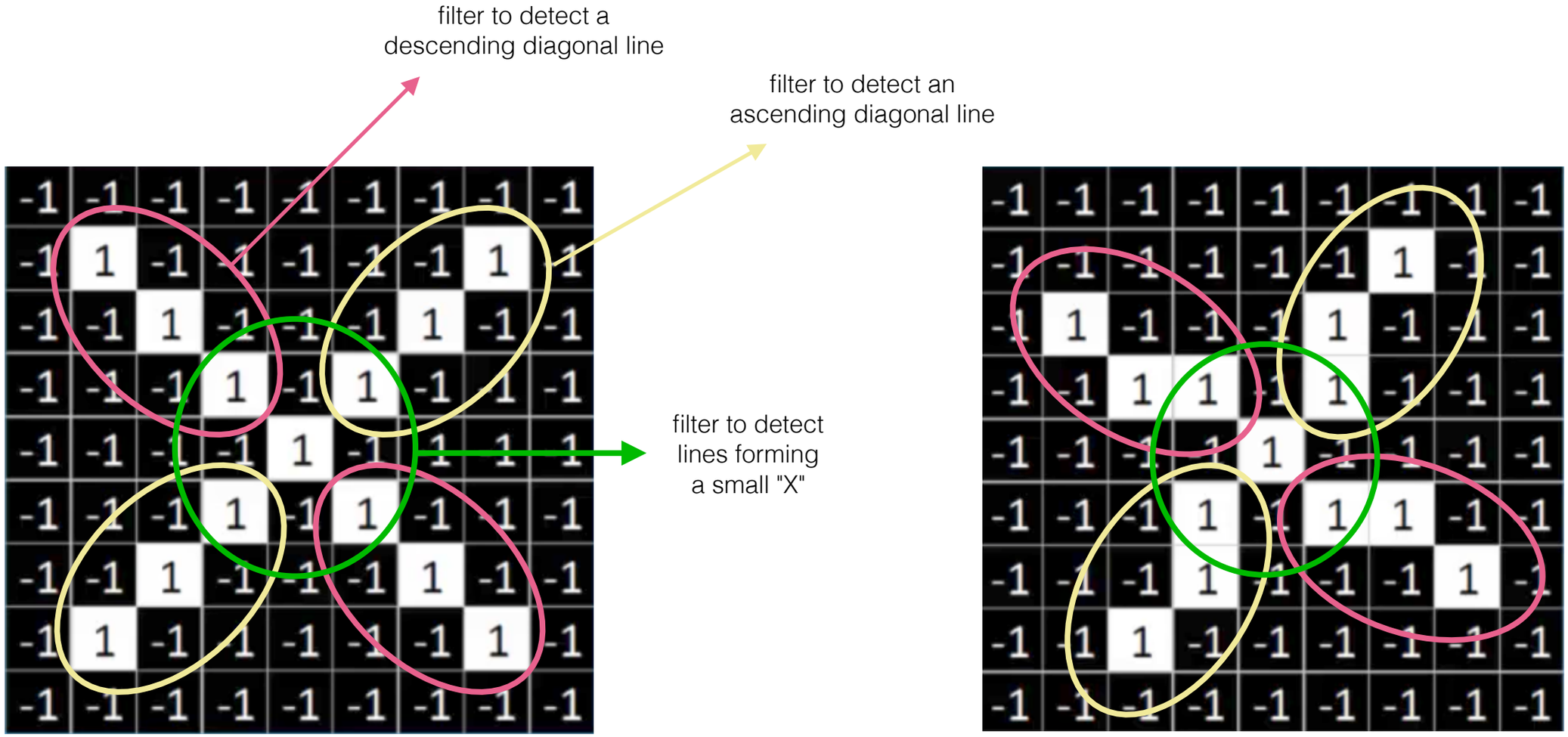
Convolutional Neural Networks





What does the network “see” as input?

Convolutional Neural Networks



What does the network “see” as input?

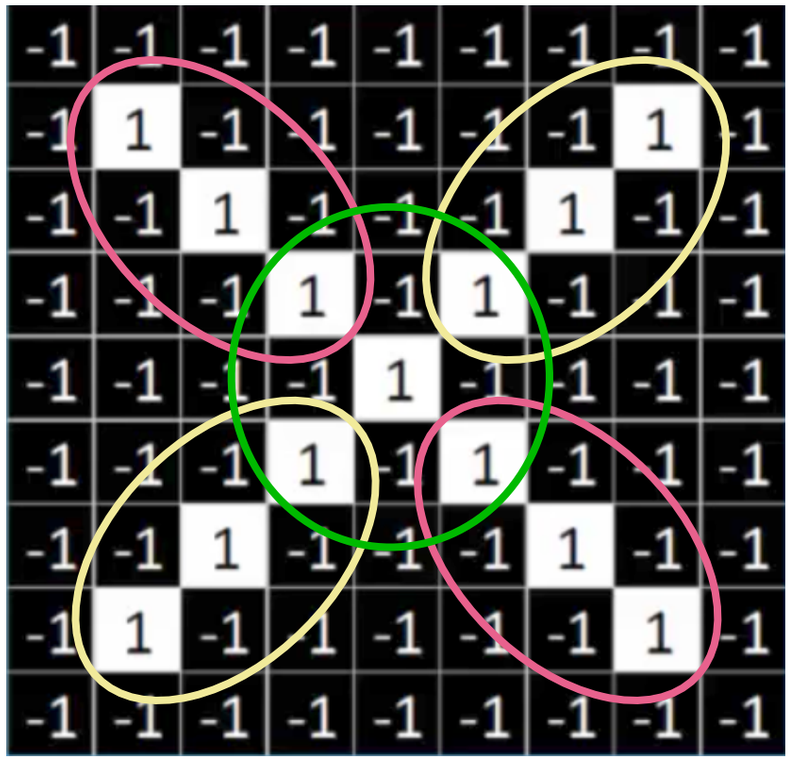
Let us assume filters that detect features/characteristics of an image to more easily identify its class

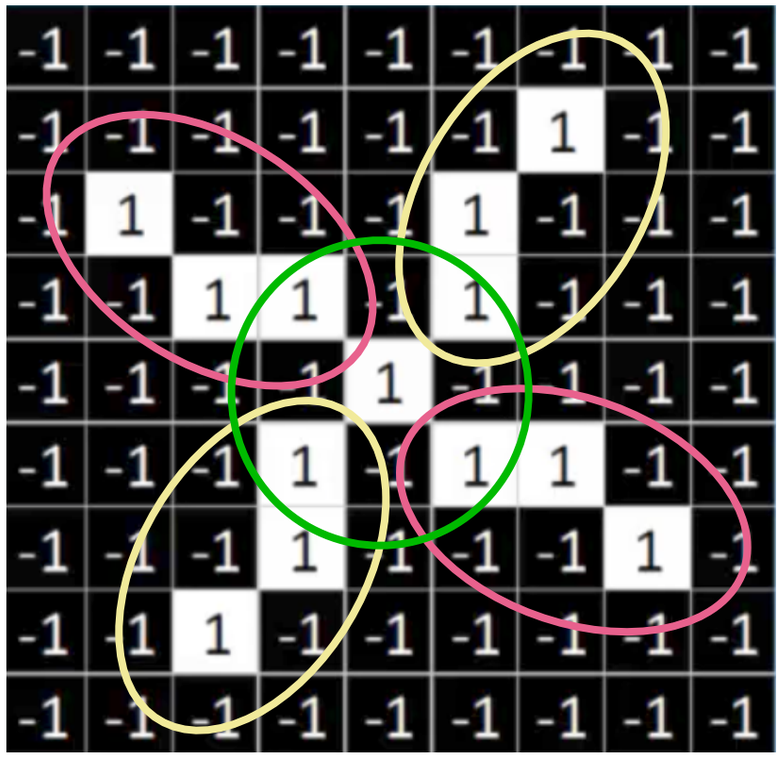
Convolutional Neural Networks

descending diagonal line

ascending diagonal line

lines forming small "X"





What does the network “see” as input?

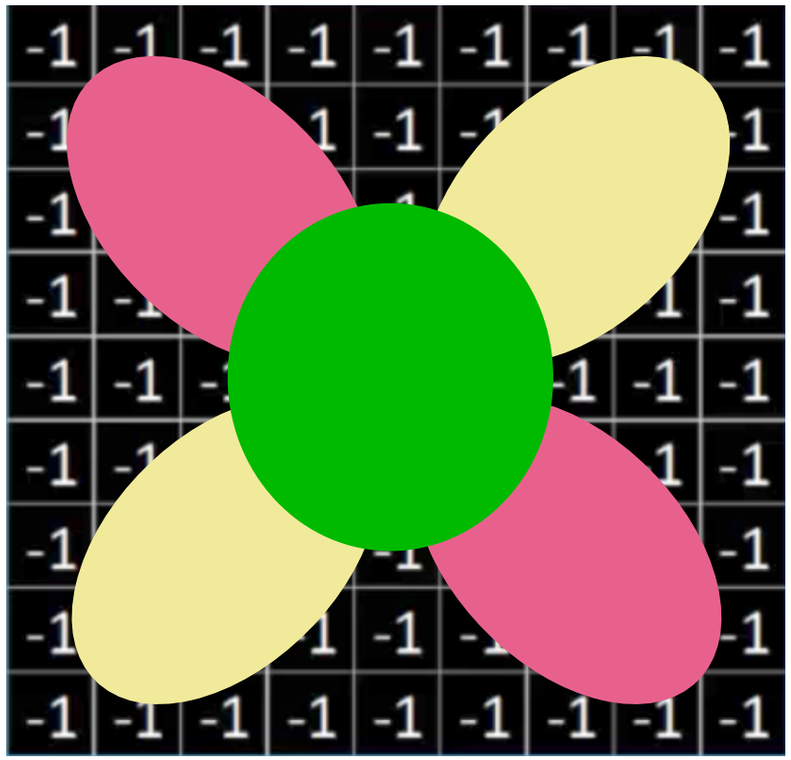
Let us assume filters that detect features/characteristics of an image to more easily identify its class

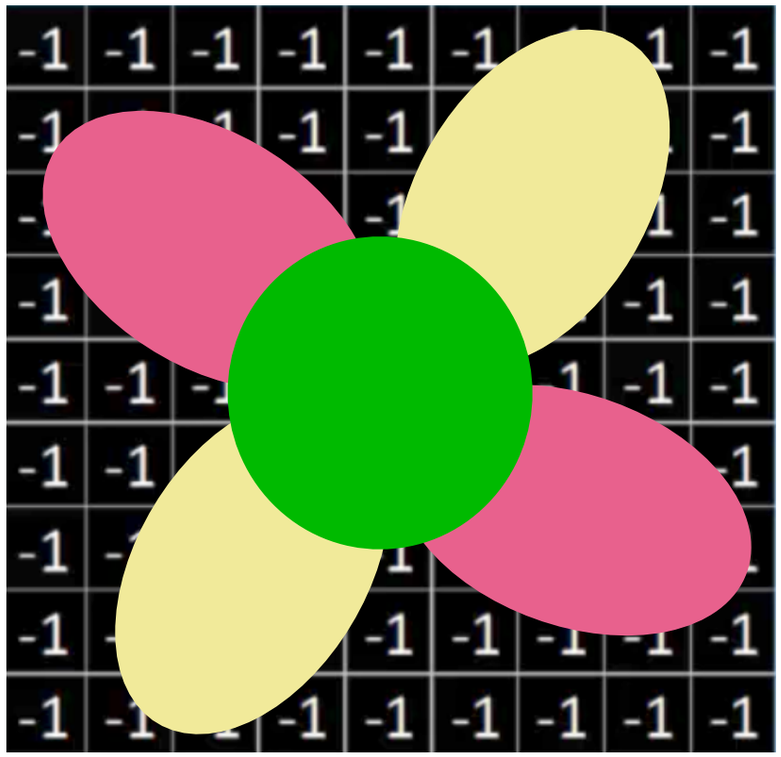
Convolutional Neural Networks

descending diagonal line

ascending diagonal line

lines forming small "X"





What does the network “see” as input?

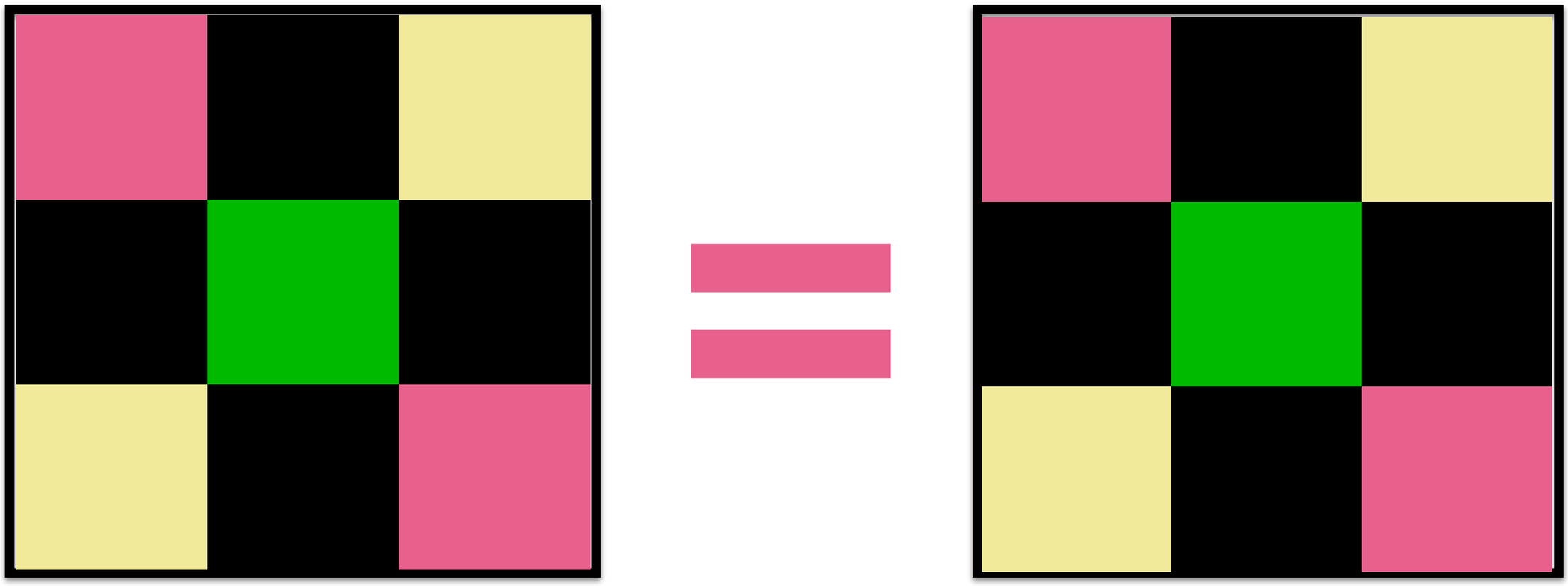
Let us assume filters that detect features/characteristics of an image to more easily identify its class

Convolutional Neural Networks

descending diagonal line

ascending diagonal line

lines forming small "X"



“Simplified" version of the images indicating the presence of different features/characteristics

Machine Learning
CMPSCI 589

Bruno C. da Silva
bsilva@cs.umass.edu

Neural Networks (pt. 4)

Numerically Checking Gradients
Heuristics to Accelerate Convergence

UMassAmherst
College of Information & Computer Sciences